

FC7xxx CAN User Manual

Rev.0.3

Revision History

Revision	Date	Changes
0.1	2023/07/14	Initial release for MCAL V0.1.0
0.3	2023/10/20	Updated for MCAL V0.3.0

Table of Contents

Chapter 1 CAN Introduction	5
1.1 Requirements.....	5
1.2 Hardware Summary.....	5
Chapter 2 Software Design	6
2.1 Rejected Requirements	6
2.2 File Structure.....	11
2.3 Macros	11
2.3.1 Macros in Can_Cfg.h	11
2.3.2 Macros in Can_GeneralTypes.h.....	16
2.3.3 Macros in Can_HwType.h	17
2.3.4 Macros in Can_Version.h	18
2.4 Enums.....	18
2.4.1 Enumerations in Can_GeneralTypes.h	18
2.4.2 Enumerations in Can_Hal.c.....	20
2.5 Typedefs	20
2.5.1 Typedefs in Can_GeneralTypes.h	20
2.6 Structures.....	21
2.6.1 Can_ConfigType.....	21
2.6.2 Can_ControllerInstanceType	21
2.6.3 Can_ControllerBaudrateConfigType.....	22
2.6.4 Can_ControllerConfigContainerType.....	22
2.6.5 Can_ControllerFdConfigType	23
2.6.6 Can_HwType	23
2.6.7 Can_IcomConfigsType.....	24
2.6.8 Can_IcomRxMessageConfigsType.....	24
2.6.9 Can_IcomRxMessageSignalType	25
2.6.10 Can_MBConfigContainerType.....	25
2.6.11 Can_MBConfigObjectType	26
2.6.12 Can_MBDlcMaxMinType	26
2.6.13 Can_PduType.....	27
2.6.14 CanHal_ConfigType	27
2.6.15 Can_RxFiFoTableIdConfigType.....	27
2.6.16 CanHal_ControllerInstanceType	28
2.6.17 CanHal_ControllerStatusType	28
2.6.18 CanHw_MbOffsetWithHoh	29
2.6.19 CanHw_Message	29

2.7	API Functions	29
2.7.1	Functions in Can.h	29
2.8	Hardware Functions	35
2.8.1	Functions in Can_Hal.h	35
2.8.2	Functions in Can_Hw.h.....	40
2.8.3	Functions in Can_Irq.c.....	60
2.9	API Sequence Diagram	61
2.9.1	Transmit Request (Single CAN Driver)	61
2.9.2	Transmit Confirmation (Interrupt Mode)	62
2.9.3	Transmit Confirmation (Polling Mode).....	63
2.9.4	Transmit Confirmation (with Buffering).....	63
Chapter 3 Tresos Configuration Items		65
3.1	Container Inclusion Relation.....	65
3.2	Containers and variables.....	66
3.2.1	CanGeneral.....	66
3.2.2	CanConfigSet.....	73
3.2.3	CommonPublishedInformation.....	111
Chapter 4 Configuration Guides.....		115
4.1	CAN Usage Common Steps	115
4.2	CAN Usage Demo	115

Chapter 1 CAN Introduction

1.1 Requirements

The design of this module follows the specifications of the CAN driver specified in AUTOSAR Classic Platform Release 4.6.0. For detailed requirements, refer to the *AUTOSAR_SWS_CANDriver*.

1.2 Hardware Summary

The FC7300 series supports eight FLEXCAN instances: FLEXCAN0-FLEXCAN7.

The FLEX Controller Area Network (FLEXCAN) module includes the following major features:

- Full implementation of the CAN FD protocol and CAN Specification 2.0, Part B
 - Standard data frames
 - Extended data frames
 - 0 to 64 bytes data length
 - Programmable bit rate
 - Content-related addressing
- Compliant with the ISO 11898-1 standard
- Flexible mailboxes configurable to store 0 to 8, 16, 32, or 64 bytes data length
- Individual Rx Mask registers per mailbox
- Listen-only mode capability
- Programmable transmission priority scheme: lowest ID, lowest buffer number, or highest priority
- Independence from the transmission medium (an external transceiver is assumed)
- Short latency time due to an arbitration scheme for high-priority messages
- Low power modes or matching with received frames (Pretended Networking)
- Transceiver delay compensation feature when transmitting CAN FD messages at faster data rates Powerful Rx FIFO ID filtering, capable of matching incoming IDs against either 128 extended, 256 standard, or 512 partial (8 bit) IDs, with up to 32 individual masking capability
- Support Pretended Networking functionality in low power: stop mode.

Chapter 2 Software Design

2.1 Rejected Requirements

Rejected Requirement 1 **SWS_Can_00242**

Description	If an off-chip CAN controller is used, the Can module shall use services of other MCAL drivers (e.g. SPI).
Rejection Reason	All current controllers are onchip.

Rejected Requirement 2 **SWS_Can_00244**

Description	The Can module shall use the synchronous APIs of the underlying MCAL drivers and shall not provide callback functions that can be called by the MCAL drivers.
Rejection Reason	All current controllers are onchip. No callback can be called by other drivers.

Rejected Requirement 3 **SWS_Can_00391**

Description	Can module implementations for off-chip CAN controllers shall include the header file Spi.h. By this inclusion, the APIs to access an external CAN controller by the SPI module [12] are included.
Rejection Reason	Controllers are onchip.

Rejected Requirement 4 **SWS_Can_00257**

Description	When the CAN hardware supports sleep mode and is triggered to transition into SLEEP state, the Can module shall set the controller to the SLEEP state from which the hardware can be woken over CAN Bus.
Rejection Reason	Controller not support sleep mode.

Rejected Requirement 5 **SWS_Can_00265**

Description	The function Can_SetControllerMode(CAN_T_SLEEP) shall set the controller into sleep mode.
Rejection Reason	Only applicable for platform support hardware wake-up. Otherwise, only logical sleep is implemented.

Rejected Requirement 6 **SWS_Can_00266**

Description	If the CAN HW does support a sleep mode, the function Can_SetControllerMode(CAN_CS_SLEEP) shall wait for a limited time until the CAN controller is in SLEEP state and it is assured that the CAN hardware is wake able. Compare to SWS_Can_00398.
Rejection Reason	Only applicable for platform support hardware wake-up. Otherwise, only logical sleep is implemented.

Rejected Requirement 7 **SWS_Can_00270**

Description	On hardware wakeup (triggered by a wake-up event from CAN bus), the CAN controller shall transition into the state STOPPED.
Rejection Reason	Platform does not support a WAKE UP mode.

Rejected Requirement 8 SWS_Can_00271

Description	On hardware wakeup (triggered by a wake-up event from CAN bus), the Can module shall call the function EcuM_CheckWakeup either in interrupt context or in the context of Can_MainFunction_Wakeup.
Rejection Reason	Platform does not support a WAKE UP mode.

Rejected Requirement 9 SWS_Can_00269

Description	The Can module shall not further process the L-PDU that caused a wake-up.
Rejection Reason	Platform does not support a WAKE UP mode.

Rejected Requirement 10 SWS_Can_00048

Description	In case of a CAN bus wake-up during sleep transition, the function Can_SetControllerMode(CAN_T_WAKEUP) shall return CAN_NOT_OK.
Rejection Reason	For HW not support on-chip wakeup, this event will not occur.

Rejected Requirement 11 SWS_Can_00274

Description	The Can module shall disable or suppress automatic bus-off recovery.
Rejection Reason	The CAN driver configuration shall allow automatic and also manual CAN bus-off recovery.

Rejected Requirement 12 SWS_CAN_00490

Description	Controllers that do not support a hardware FIFO often provide the capabilities to implement a shadow buffer mechanism, where additional hardware objects take over when the primary hardware object is busy. The number of hardware objects is configured via "CanHwObjectCount".
Rejection Reason	Hardware support FIFO, so this requirement is not applicable.

Rejected Requirement 13 SWS_Can_00364

Description	If the ISR for wakeup events is called, it shall call EcuM_CheckWakeup in turn. The parameter passed to EcuM_CheckWakeup shall be the ID of the wakeup source referenced by the CanWakeupSourceRef configuration parameter.
Rejection Reason	Only for platform support wake up

Rejected Requirement 14 SWS_Can_00294

Description	The function Can_SetControllerMode shall disable the wake-up interrupt, while checking the wake-up status.
Rejection Reason	Platform does not support a WAKE UP mode.

Rejected Requirement 15 SWS_Can_00361

Description	The function Can_CheckWakeup shall check if the requested CAN controller has detected a wakeup. If a wakeup event was successfully detected, reporting shall be done to EcuM via API EcuM_SetWakeupEvent.
Rejection Reason	Our SoC does not support wakeup.

Rejected Requirement 16 SWS_Can_00485

Description	The function Can_CheckWakeup shall be pre compile time configurable On/Off by the configuration parameter: CanWakeupFunctionalityAPI.
Rejection Reason	Our SoC does not support wakeup.

Rejected Requirement 17 SWS_Can_00362

Description	If development error detection for the Can module is enabled: The function Can_CheckWakeup shall raise the error CAN_E_UNINIT if the driver is not yet initialized.
Rejection Reason	Our SoC does not support wakeup.

Rejected Requirement 18 SWS_Can_00363

Description	If development error detection for the Can module is enabled: The function Can_CheckWakeup shall raise the error CAN_E_PARAM_CONTROLLER if the parameter Controller is out of range.
Rejection Reason	Our SoC does not support wakeup.

Rejected Requirement 19 SWS_Can_00445

Description	Can driver shall use the following APIs provided by Icu driver, to enable and disable the wakeup event notification:Icu_EnableNotificationIcu_DisableNotification
Rejection Reason	Not implemented for OnChip platform

Rejected Requirement 20 SWS_Can_00446

Description	Icu_EnableNotification shall be called when "external" Can controllers have been transitioned to SLEEP state (CANIF_CS_SLEEP).
Rejection Reason	Not implemented for OnChip platform

Rejected Requirement 21 SWS_Can_00447

Description	Icu_DisableNotification shall be called when "external" Can controllers have been transitioned to STOPPED state (CANIF_CS_STOPPED).
Rejection Reason	All controllers are on chip. Our SoC does not support wakeup.

Rejected Requirement 22 SWS_Can_00110

Description	There is no requirement regarding the execution order of the CAN main processing functions.
--------------------	---

Rejection Reason	this is not a requirement
-------------------------	---------------------------

Rejected Requirement 23 SWS_Can_00228

Description	Service name	Can_MainFunction_Wakeu
	Syntax	void Can_MainFunction_Wakeup(void)
	Service ID[hex]	0x0a
	Description	This function performs the polling of wake-up events that are configured statically as "to be polled".
Rejection Reason	Not implemented for platform, not support hardware wake-up	

Rejected Requirement 24 SWS_Can_00112

Description	The function Can_MainFunction_Wakeup shall perform the polling of wake-up events that are configured statically as 'to be polled'.
Rejection Reason	Not implemented for platform, does not support hardware wake-up.

Rejected Requirement 25 SWS_Can_00185

Description	The Can module may implement the function Can_MainFunction_Wakeup as empty define in case no polling at all is used.
Rejection Reason	Our SoC does not support wakeup.

Rejected Requirement 26 ECUC_Can_00430

Description	SWS Item	ECUC_Can_00430		
	Name	CanSupportTTCANRef		
	Parent Container	WdgGeneral		
	Description	The parameter refers to CanIfSupportTTCAN parameter in the CAN Interface Module configuration. The CanIfSupportTTCAN parameter defines whether TTCAN is supported.		
	Multiplicity	1		
	Type	Reference to [CanIfPrivateCfg]		
	Post-Build Variant Value	false		
	Value Configuration Class	Pre-compile time	X	All Variants
		Link time	–	
		Post-build time	–	
	Scope / Dependency	scope: ECU		
	Included Containers			
Container Name	Multiplicity	Scope / Dependency		

	CanIcomGeneral	0..1	This container contains the general configuration parameters of the ICOM Configuration.
	CanMainFunctionRWPeriods	0..*	This container contains the parameter for configuring the period for cyclic call to Can_MainFunction_Read or Can_MainFunction_Write depending on the referring item.
Rejection Reason	This is a HW limitation, some platform does not support TtCan feature.		

Rejected Requirement 27 ECUC_Can_00382

Description	SWS Item	ECUC_Can_00382		
	Name	CanControllerBaseAddress		
	Description	Specifies the CAN controller base address.		
	Multiplicity	1		
	Type	EcucIntegerParamDef		
	Range	0 .. 4294967295		
	Default value	–		
	Post-Build Variant Value	false		
	Value Configuration Class	Pre-compile time	X	All Variants
		Link time	–	
		Post-build time	–	
Scope / Dependency	scope: local			
Rejection Reason	Controller base address is taken from Base module.			

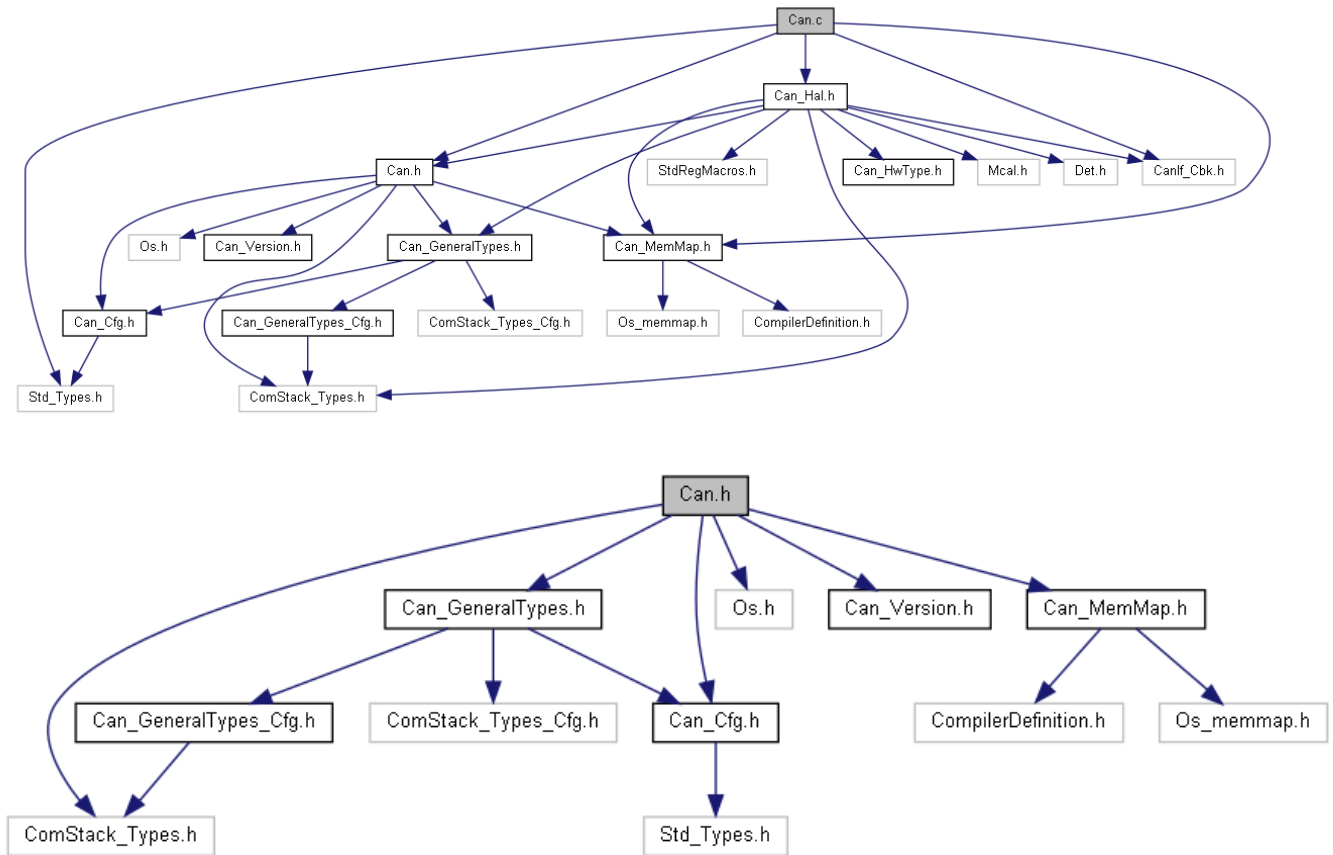
Rejected Requirement 28 ECUC_Can_00466

Description	SWS Item	ECUC_Can_00466		
	Name	CanWakeupFunctionalityAPI		
	Description	Adds / removes the service Can_CheckWakeup() from the code. True: Can_CheckWakeup can be used. False: Can_CheckWakeup cannot be used.		
	Multiplicity	1		
	Type	EcucBooleanParamDef		
	Default value	false		
	Post-Build Variant Value	false		
	Value Configuration Class	Pre-compile time	X	All Variants
		Link time	–	
		Post-build time	–	
	Scope / Dependency	scope: local dependency: H/W should support the wakeup functionality to enable this parameter.		

Rejection Reason

Shall be supported only by platforms which include hardware wake-up supported.

2.2 File Structure



2.3 Macros

2.3.1 Macros in Can_Cfg.h

- `#define CAN_VENDOR_ID_PCCFG_CUR 174`
- `#define CAN_MODULE_ID_PCCFG_CUR 80`
- `#define CAN_AR_RELEASE_MAJOR_VERSION_PCCFG_CUR 4`
- `#define CAN_AR_RELEASE_MINOR_VERSION_PCCFG_CUR 6`
- `#define CAN_AR_RELEASE_REVISION_VERSION_PCCFG_CUR 0`
- `#define CAN_SW_MAJOR_VERSION_PCCFG_CUR 0`
- `#define CAN_SW_MINOR_VERSION_PCCFG_CUR 3`
- `#define CAN_SW_PATCH_VERSION_PCCFG_CUR 0`

- #define **CAN_WAKEUP_ENABLE** (STD_OFF)
- #define **CAN_MAXMASKCOUNT** (Can_HwHandleType)65535U
- #define **CanConf_CanController_CanController_0** ((uint8)0U)
- #define **CanConf_CanController_CanController_1** ((uint8)1U)
- #define **CanHardwareObject_0** 0U
- #define **CanHardwareObject_1** 1U
- #define **CanHardwareObject_2** 2U
- #define **CanHardwareObject_3** 3U
- #define **CanHardwareObject_4** 4U
- #define **CanHardwareObject_5** 5U
- #define **CanHardwareObject_6** 6U
- #define **CAN_LPDU_NOTIFICATION_ENABLE** (STD_OFF)
- #define **CAN_EXTENDEDID** (STD_ON)
- #define **CAN_SET_BAUDRATE_API** (STD_OFF)
- #define **CAN_DEV_ERROR_DETECT** (STD_OFF)
- #define **CAN_MULTIPLE_INTERRUPTS_SUPPORT** (STD_ON)
- #define **CAN_MBCOUNTEXTENSION** (STD_ON)
- #define **CAN_VERSION_INFO_API** (STD_ON)
- #define **CAN_MULTIPLEXED_TRANSMISSION** (STD_ON)
- #define **CAN_HW_TRANSMIT_CANCELLATION** (STD_OFF)
- #define **CAN_API_ENABLE_ABORT_MB** (STD_OFF)
- #define **CAN_ABORT_ONLY_ONE_MB** (STD_OFF)

- `#define CAN_INSTANCE 0U`
- `#define CAN_TRIGGER_TRANSMIT_EN (STD_OFF)`
- `#define CAN_MIX_MB_SUPPORT (STD_OFF)`
- `#define CAN_MEMORY_ECC_SUPPORT (STD_OFF)`
- `#define CAN_MAXMBCOUNT_0 7U`
- `#define CAN_MAXMB_SUPPORTED 32U`
- `#define CAN_CONTROLLER__MAXMB_CONFIGURED 32`
- `#define CAN_MAXMB_CONFIGURED 32U`
- `#define CAN_MAXCTRL_SUPPORTED 6U`
- `#define CAN_MAXCONTROLLERCOUNT_0 1U`
- `#define CAN_MAXCTRL_CONFIGURED 1U`
- `#define CAN_0_TXINT_SUPPORTED (STD_OFF)`
- `#define CAN_TXPOLL_SUPPORTED (STD_ON)`
- `#define CAN_TXINT_SUPPORTED (STD_OFF)`
- `#define CAN_0_RXINT_SUPPORTED (STD_ON)`
- `#define CAN_RXPOLL_SUPPORTED (STD_OFF)`
- `#define CAN_RXINT_SUPPORTED (STD_ON)`
- `#define CAN_TX_RX_INTERRUPT_SUPPORTED (STD_ON)`
- `#define CAN_0_BUSOFFINT_SUPPORTED (STD_ON)`
- `#define CAN_1_BUSOFFINT_SUPPORTED (STD_OFF)`
- `#define CAN_BUSOFFPOLL_SUPPORTED (STD_ON)`
- `#define CAN_BUSOFFINT_SUPPORTED (STD_ON)`

- `#define CAN_MAXFILTERCOUNT_0 4U`
- `#define CAN_CFGSET0_MAX_BAUDRATE_FC_0_0 1U`
- `#define CAN_CFGSET0_MAX_BAUDRATE_FC_1_1 2U`
- `#define CAN_FD_MODE_ENABLE (STD_OFF)`
- `#define CAN_CONTROLLER_FD_ISO_CANFD (STD_OFF)`
- `#define CAN_CONTROLLER_FD_PREXCEN (STD_ON)`
- `#define CAN_CONTROLLER_FD_EDFLTDIS (STD_ON)`
- `#define CAN_LEGACY_FIFO_MAX_TABLEID_0 8U`
- `#define CAN_LEGACY_RXFIFO_ENABLE (STD_ON)`
- `#define CAN_LEGACY_RXFIFO_EVENT_UNIFIED (STD_ON)`
- `#define CAN_ENHANCED_RXFIFO_ENABLE (STD_OFF)`
- `#define CAN_ERROR_NOTIFICATION_ENABLE (STD_ON)`
- `#define CAN_CAN0_CONTROLLER_ID ((uint8)0U)`
- `#define FLEXCAN_0_INDEX ((uint8)0U)`
- `#define CAN_0_LEGACY_FIFO_EN (STD_ON)`
- `#define CAN_0_FD_EN (STD_OFF)`
- `#define CAN_0_ISOFD_EN (STD_ON)`
- `#define CAN_0_ERROR_NOTIFICATION_ENABLE (STD_ON)`
- `#define CAN_0_MB_BUF_SIZE 8U`
- `#define CAN_CAN1_CONTROLLER_ID ((uint8)1U)`
- `#define FLEXCAN_1_INDEX ((uint8)1U)`
- `#define CAN_1_LEGACY_FIFO_EN (STD_OFF)`

- #define **CAN_1_FD_EN** (STD_OFF)
- #define **CAN_1_ISOFD_EN** (STD_ON)
- #define **CAN_1_ERROR_NOTIFICATION_ENABLE** (STD_ON)
- #define **CAN_1_MB_BUF_SIZE** 8U
- #define **CAN_ISROPTCODESIZE** (STD_OFF)
- #define **CAN_TIMEOUT_DURATION** 1.0E8F
- #define **CAN_TIMEOUT_COUNTER_NAME** SystemCounterOsCore0
- #define **CAN_TIMEOUT_COUNTER_TICK2NS()** OS_TICKS2NS_SystemCounterOsCore0(1U)
- #define **CAN_PUBLIC_ICOM_SUPPORT** (STD_OFF)
- #define **CAN_MAINFUNCTION_PERIOD** (0.001F)
- #define **CAN_MAINFUNCTION_PERIOD_READ** (0.001F)
- #define **CAN_MAINFUNCTION_MULTIPLE_READ** (STD_OFF)
- #define **CAN_MAINFUNCTION_PERIOD_WRITE** (0.001F)
- #define **CAN_MAINFUNCTION_MULTIPLE_WRITE** (STD_OFF)
- #define **CAN_MAINFUNCTION_MODE_PERIOD** 0U
- #define **CAN_BCC_SUPPORT_ENABLE** (STD_ON)
- #define **CAN_MAX_OBJECT_ID** ((uint32)7U)
- #define **CAN_CONTROLLER_REFERENCE_COUNTER** 6
- #define **CAN_MB_CONTROLLER_1** 32U
- #define **CAN_MB_CONTROLLER_2** 32U
- #define **CAN_MB_CONTROLLER_3** 32U
- #define **CAN_MB_CONTROLLER_4** 32U

- #define **CAN_MB_CONTROLLER_5** 32U
- #define **CAN_MB_CONTROLLER_6** 32U

2.3.2 Macros in Can_GeneralTypes.h

- #define **CAN_MAX_DLC** (8)
- #define **CANFD_MAX_DLC** (64)
- #define **CAN_STD_MAX_CANID** ((Can_IdType)0x07FFU)
- #define **CAN_EXT_MAX_CANID** (0x1FFFFFFFUL)
- #define **CAN_EXT_CANID_FLAG** (0x80000000UL)
- #define **CAN_E_PARAM_POINTER** ((uint8)0x01U)
- #define **CAN_E_PARAM_HANDLE** ((uint8)0x02U)
- #define **CAN_E_PARAM_DATA_LENGTH** ((uint8)0x03U)
- #define **CAN_E_PARAM_CONTROLLER** ((uint8)0x04U)
- #define **CAN_E_UNINIT** ((uint8)0x05U)
- #define **CAN_E_TRANSITION** ((uint8)0x06U)
- #define **CAN_E_PARAM_BAUDRATE** ((uint8)0x07U)
- #define **CAN_E_ICOM_CONFIG_INVALID** ((uint8)0x08U)
- #define **CAN_E_INIT_FAILED** ((uint8)0x09U)
- #define **CAN_E_DATA_LOST** ((uint8)0x01U) /* Receive CAN message is lost */
- #define **CAN_DET_SID_INIT** ((uint8)0x00U)
- #define **CAN_DET_SID_GET_VERSION_INFO** ((uint8)0x07U)
- #define **CAN_DET_SID_DEINIT** ((uint8)0x10U)
- #define **CAN_DET_SID_SET_BAUDRATE** ((uint8)0x0FU)

- `#define CAN_DET_SID_SET_CONTROLLER_MODE ((uint8)0x03U)`
- `#define CAN_DET_SID_DISABLE_CONTROLLER_INTERRUPTS ((uint8)0x04U)`
- `#define CAN_DET_SID_ENABLE_CONTROLLER_INTERRUPTS ((uint8)0x05U)`
- `#define CAN_DET_SID_CBK_CHECK_WAKEUP ((uint8)0x0BU)`
- `#define CAN_DET_SID_GET_CONTROLLER_ERROR_STATE ((uint8)0x11U)`
- `#define CAN_DET_SID_GET_CONTROLLER_MODE ((uint8)0x12U)`
- `#define CAN_DET_SID_WRITE ((uint8)0x06U)`
- `#define CAN_DET_SID_MAIN_FUNCTION_WRITE ((uint8)0x01U)`
- `#define CAN_DET_SID_MAIN_FUNCTION_READ ((uint8)0x08U)`
- `#define CAN_DET_SID_MAIN_FUNCTION_BUS_OFF ((uint8)0x09U)`
- `#define CAN_DET_SID_MAIN_FUNCTION_WAKEUP ((uint8)0x0AU)`
- `#define CAN_DET_SID_MAIN_FUNCTION_MODE ((uint8)0x0CU)`
- `#define CAN_DET_SID_SET_ICOM_CONFIG ((uint8)0x21U)`

2.3.3 Macros in Can_HwType.h

- `#define CAN_HW_REG_WORD_LEN 0x04u`
- `#define CAN_WRITE_REG8(address, value) ((*volatile uint8*)(address))= (value)`
- `#define CAN_WRITE_REG32(address, value) ((*volatile uint32*)(address))= (value)`
- `#define CAN_READ_REG8(address) (*(volatile uint8*)(address))`
- `#define CAN_READ_REG32(address) (*(volatile uint32*)(address))`
- `#define CAN_AREAD_REG32(address, i) (CAN_READ_REG32((address)+((uint32)((i)<<2U))))`
- `#define CAN_BIT_CLR_REG8(address, mask) ((*volatile uint8*)(address))&= (~(uint8)(mask)))`
- `#define CAN_BIT_CLR_REG32(address, mask) ((*volatile uint32*)(address))&= (~(uint32)(mask)))`

- `#define CAN_BIT_GET_REG32(address, mask) ((*(volatile uint32*)(address)) & ((uint32)(mask)))`
- `#define CAN_BIT_SET_REG8(address, mask) ((* (volatile uint8*)(address)) |= ((uint32)(mask)))`
- `#define CAN_BIT_SET_REG32(address, mask) ((* (volatile uint32*)(address)) |= ((uint32)(mask)))`
- `#define CAN_RMW_REG8(address, mask, value) (CAN_WRITE_REG8((address), ((CAN_READ_REG8(address) & ((uint8)~((uint8)(mask)))) | (value))))`
- `#define CAN_RMW_REG32(address, mask, value) (CAN_WRITE_REG32((address), ( (CAN_READ_REG32(address) & ( (uint32)~((uint32)(mask)) ) ) | (uint32)(value) )))`

2.3.4 Macros in Can_Version.h

- `#define CAN_VENDOR_ID 174`
- `#define CAN_MODULE_ID 80`
- `#define CAN_AR_RELEASE_MAJOR_VERSION 4`
- `#define CAN_AR_RELEASE_MINOR_VERSION 6`
- `#define CAN_AR_RELEASE_REVISION_VERSION 0`
- `#define CAN_SW_MAJOR_VERSION 0`
- `#define CAN_SW_MINOR_VERSION 3`
- `#define CAN_SW_PATCH_VERSION 0`

2.4 Enums

2.4.1 Enumerations in Can_GeneralTypes.h

2.4.1.1 Can_HwObjType

Enumeration	Can_HwObjType	
Description	Can MB Hardware Object Type.	
Values	Value	Description
	CAN_RECEIVE = 0U	N/A
	CAN_TRANSMIT	N/A

2.4.1.2 Can_IdTypeType

Enumeration	Can_IdTypeType
Description	CAN ID type.

Values	Value	Description
	CAN_EXTENDED = 0U	N/A
	CAN_STANDARD	N/A
	CAN_MIXED	N/A

2.4.1.3 Can_StatusType

Enumeration	Can_StatusType	
Description	Can controller initial status.	
Values	Value	Description
	CAN_UNINIT = 0U,	N/A
	CAN_READY	N/A

2.4.1.4 Can_ControllerStateType

Enumeration	Can_ControllerStateType	
Description	Can Controller Run status.	
Values	Value	Description
	CAN_CS_UNINIT = 0U,	N/A
	CAN_CS_STARTED	N/A
	CAN_CS_STOPPED	N/A
	CAN_CS_SLEEPED	N/A

2.4.1.5 Can_ReturnType

Enumeration	Can_ReturnType	
Description	CAN Function Call Result.	
Values	Value	Description
	CAN_OK = 0U	N/A
	CAN_NOT_OK	N/A
	CAN_BUSY	N/A

2.4.1.6 Can_ErrorStateType

Enumeration	Can_ErrorStateType	
Description	Error states of a CAN controller.	
Values	Value	Description
	CAN_ERRORSTATE_ACTIVE = 0U	N/A
	CAN_ERRORSTATE_PASSIVE	N/A
	CAN_ERRORSTATE_BUSOFF	N/A

2.4.1.7 Can_IcomSignalOperationType

Enumeration	Can_IcomSignalOperationType	
Description	Pretended Network Signal Operation Type.	
Values	Value	Description

	AND = 0U	N/A
	EQUAL	N/A
	GREATER	N/A
	SMALLER	N/A
	XOR	N/A

2.4.1.8 Can_IcomIdOperationType

Enumeration	Can_IcomIdOperationType	
Description	Pretended Network ID operation Type.	
Values	Value	Description
	EXACTLY = 0U	N/A
	GREATER_MINNUM	N/A
	SMALLER_MAXNUM	N/A
	INSIDE_RANGE	N/A

2.4.2 Enumerations in Can_Hal.c

2.4.2.1 CanHal_InterruptStateType

Enumeration	CanHal_InterruptStateType	
Description	Can Interrupt Status.	
Values	Value	Description
	CAN_INTERRUPT_DISABLED = 0U	N/A
	CAN_INTERRUPT_ENABLED	N/A

2.4.2.2 CanHal_IcomStateType

Enumeration	CanHal_IcomStateType	
Description	N/A	
Values	Value	Description
	CAN_ICOM_DEACTIVATED = 0U	N/A
	CAN_ICOM_ACTIVATED	N/A

2.5 Typedefs

2.5.1 Typedefs in Can_GeneralTypes.h

- typedef void(* Can_PCallBackType) (void)
- typedef uint16 Can_IdType
- typedef uint8 Can_HwHandleType
- typedef uint32 Can_RxEnhancedFiFoTableIdConfigType

2.6 Structures

2.6.1 Can_ConfigType

Structure	Can_ConfigType
Description	This is the type of the external data structure containing the overall initialization data for the CAN driver and SFR settings affecting all controllers. Furthermore it contains pointers to controller configuration structures. The contents of the initialization data structure are CAN hardware specific.
Diagram	<pre> graph TD Can_ConfigType --> Can_RxFifoTableIdConfigType Can_ConfigType --> pRxFifoTableIdConfig Can_ConfigType --> Can_MBCfgContainerType Can_ConfigType --> tMBCfgContainer Can_ConfigType --> Can_ControllerConfigContainerType Can_ConfigType --> tControllerContainer Can_ConfigType --> CanHal_ConfigType Can_ConfigType --> tHwCfgContainer Can_ConfigType --> Can_MBCfgContainerType Can_ConfigType --> tMBCfgContainer Can_ConfigType --> Can_HwHandleType Can_ConfigType --> u16CanFirstHTHIndex Can_ControllerFdConfigType --> ControllerFD Can_ControllerBaudrateConfigType --> pControllerBaudCfgsPtr Can_ControllerInstanceType --> pInstanceCfgsPtr Can_IcomRxMessageSignalType --> pCanIcomRxMessageSignalCfgs Can_IcomRxMessageConfigType --> pCanIcomRxMessageCfgs Can_IcomConfigType --> pCanIcomCfgs CanHal_ControllerInstanceType --> pControllerInstances </pre>
Data Fields	- const Can_IdType* const pFilterMasks CAN ID Filter Array - const Can_RxFifoTableIdConfigType* const pRxFifoTableIdConfig CAN Legacy Rx FIFO Configuration - const Can_ControllerConfigContainerType tControllerContainer CAN Controller Configuration container - const CanHal_ConfigType tHwCfgContainer CAN Hardware register configuration - const Can_MBCfgContainerType tMBCfgContainer CAN Hardware Object Configuration container - const Can_HwHandleType u16CanFirstHTHIndex CAN First Hardware transmit handler index

2.6.2 Can_ControllerInstanceType

Structure	Can_ControllerInstanceType
Description	Can Controller Instance Configuration.
Diagram	<pre> graph TD Can_ControllerInstanceType --> pControllerBaudCfgsPtr pControllerBaudCfgsPtr --> Can_ControllerBaudrateConfigType Can_ControllerBaudrateConfigType --> ControllerFD ControllerFD --> Can_ControllerFdConfigType </pre>
Data Fields	- const Can_PCallBackType Can_RxFifoOverflowNotification Pointer to Rx FIFO Overflow notification function - const Can_PCallBackType Can_RxFifoWarningNotification Pointer to Rx FIFO Warning notification function - const uint16Can_u16RxFifoHrhFirst First HRH used for RX FIFO - const Can_ControllerBaudrateConfigType* const pControllerBaudCfgsPtr Baudrate Configuration array

	• const uint32 u32MBBlockSize Message Buffer Data Size for RAM regions • const uint32 u32Options Controller Options • const uint32u32RxFifoGlobalMask Legacy Rx Fifo Global Mask value • const uint8 u8DefaultBaudIndex Default BaudRate configured • const uint8 u8MaxBaudCnt No. of BaudRate configured for current controller • const uint8 u8MaxMBCnt Max Message Buffer Number In Current Can • const uint8 u8RxFifoUsedMb Number of MBs used by Rx Fifo
--	---

2.6.3 Can_ControllerBaudrateConfigType

Structure	Can_ControllerBaudrateConfigType
Description	Ftu common configuration structure type.
Diagram	<pre> graph BT A[Can_ControllerBaudrateConfigType] -.-> ControllerFD B[Can_ControllerFdConfigType] </pre>
Data Fields	• const Can_ControllerFdConfigTypeControllerFD FD settings. • const uint16 u16ControllerBaud Baudrate. • const uint16 u16ControllerBaudRateConfigID Baudrate Configuration ID. • const uint16 u16CtrlPrescaler Prescaler in CTRL. • const uint32 u32CbtReg CBT register value. • const uint8 u8TASD TASD value.

2.6.4 Can_ControllerConfigContainerType

Structure	Can_ControllerConfigContainerType
Description	Can Controller Container.

Diagram	<pre> graph TD A[Can_ControllerFdConfigType] -.-> ControllerFD B[Can_ControllerBaudrateConfigType] B -.-> pControllerBaudCfgsPtr C[Can_ControllerInstanceType] C -.-> pInstanceCfgsPtr D[Can_ControllerConfigContainerType] style D fill:#ccc </pre>
Data Fields	- const Can_ControllerInstanceType* const pInstanceCfgsPtr Controller Instance Array. - const uint16 u16InstanceCfgCnt Controller Instance Count.

2.6.5 Can_ControllerFdConfigType

Structure	Can_ControllerFdConfigType
Description	Can FD Configuration.
Diagram	N/A
Data Fields	- const uint32 u32CanControllerTDCOFF TDCOFF value. - const uint32 u32CanControllerTxBRS Enable BRS. - const uint32 u32CanFdBaud Can FD Baudrate. - const uint32 u32CanFdCbtReg FDCBT Register value. - const uint32 u32CanFdCTRL2Reg CTRL2 Register value. - const uint32 u32CanFdEN Can FD Enable.

2.6.6 Can_HwType

Structure	Can_HwType
Description	This type defines a data structure which clearly provides an Hardware Object Handle including its corresponding CAN Controller and therefore CanDrv as well as the specific CanId.
Diagram	N/A
Data Fields	- Can_IdType CanId Standard/Extended CAN ID of CAN L-PDU. - uint8 ControllerId ControllerId provided by CanIf clearly identify the corresponding controller.

- Can_HwHandleType Hoh
ID of the corresponding Hardware Object Range.

2.6.7 Can_IcomConfigsType

Structure	Can_IcomConfigsType
Description	Pretended Network Configuration Type.
Diagram	<pre> graph BT A[Can_IcomConfigsType] -.-> pCanIcomRxMessageConfigs B[Can_IcomRxMessageConfigsType] B -.-> pCanIcomRxMessageSignalConfigs C[Can_IcomRxMessageSignalType] </pre>
Data Fields	• const boolean CanIcomWakeOnBusOff Wake on Bus Off enable. • const Can_IcomRxMessageConfigsType* pCanIcomRxMessageConfigs Rx Message Config array. • const uint8 u8CanIcomConfigId Config ID. • const uint8 u8NumberCanIcomRxMessage Rx Message Config Number.

2.6.8 Can_IcomRxMessageConfigsType

Structure	Can_IcomRxMessageConfigsType
Description	Pretended Network Rx Message Configuration Type.
Diagram	<pre> graph BT A[Can_IcomRxMessageConfigsType] -.-> pCanIcomRxMessageSignalConfigs B[Can_IcomRxMessageSignalType] </pre>
Data Fields	• const boolean CanFiltPayload Filt Payload. • const uint32 CanIcomCounterValue Counter value. • const Can_IcomIdOperationType CanIcomIdOperation Id operation. • const uint32 CanIcomMessageId Message ID. • const uint32 CanIcomMessageIdMask ID Mask. • const uint32 CanIcomMissingMessageTimerValue

	Missing Message timer value. - const Can_IcomRxMessageSignalType* pCanIcomRxMessageSignalConfigs Rx Message Signal Configuration Array. - const uint8 u8NumCanIcomRxMessageSignal Rx Message Signal Number.
--	---

2.6.9 Can_IcomRxMessageSignalType

Structure	Can_IcomRxMessageSignalType
Description	Pretended Network Rx Message Signal Type.
Diagram	N/A
Data Fields	- const uint32 CanIcomSignalMaskHigh Pretended Network Signal Mask High. - const uint32 CanIcomSignalMaskLow Pretended Network Signal Mask Low. - const Can_IcomSignalOperationType CanIcomSignalOperation Pretended Network Signal Operation. - const uint32 CanIcomSignalRef Reference it is not available for now. It may be referenced from an upper layer. - const uint32 CanIcomSignalValueHigh Pretended Network Signal Value High. - const uint32 CanIcomSignalValueLow Pretended Network Signal Value Low. - const uint8 DLCHighValue Pretended Network Data Length Value High. - const uint8 DLCLowValue Pretended Network Data Length Value Low.

2.6.10 Can_MBConfigContainerType

Structure	Can_MBConfigContainerType
Description	Message Buffer Container.
Diagram	<pre> graph BT A[Can_MBConfigContainerType] -.-> pMsgBufCfgsPtr B[Can_MBConfigObjectType] </pre>
Data Fields	- const Can_MBConfigObjectType* const pMsgBufCfgsPtr Message Buffer Array. - const uint16 u16MsgBufCfgCnt Message Buffer Count.

2.6.11 Can_MBConfigObjectType

Structure	Can_MBConfigObjectType
Description	Message Buffer Configuration.
Diagram	N/A
Data Fields	• const boolean CanTriggerTransmitEnable Trigger transmit enable. • Can_IdTypeType eldType Can ID Type, Standard, Extended, Mixed. • Can_HwObjType eMBType Message Buffer Type, Transmit, Receive. • uint16 u16IdMaskIndex Id Mask Table Index. • const uint16 u16MBOffsetAddr MB Offset in Current Can. • uint32 u32HWObjID MB Logical Index configured in EB. • uint32 u32MessageId CAN ID. • const uint8 u8CanMainFuncRWPeriodRef Multi-Polling Id. • const uint8 u8ControllerId Controller ID configured in EB. • uint8 u8FdPaddingValue Can FD Pad value. • const uint8 u8HWMBIndex MB Index In Current Can. • const uint8 u8LocalPriority Can ID Priority. • const uint8 u8MBPayloadLength MB Data Length. • const boolean bInterruptEn MB interrupt enable.

2.6.12 Can_MBDlcMaxMinType

Structure	Can_MBDlcMaxMinType
Description	CAN Message Buffer Data Length Range.
Diagram	N/A
Data Fields	• const uint8 u8DlcMax Max Data Length. • const uint8 u8DlcMin Min Data Length.

2.6.13 Can_PduType

Structure	Can_PduType
Description	This type unites PduId (swPduHandle), SduLength (length), SduData (sdu), and CanId (id) for any CAN L-SDU.
Diagram	N/A
Data Fields	- Can_IdType id Can ID. - uint8 length Data Length. - uint8* sdu Point to Data Array. - PduIdType swPduHandle Pdu Handler.

2.6.14 CanHal_ConfigType

Structure	CanHal_ConfigType
Description	Can Controller Hardware Instance Configuration.
Diagram	<pre> graph BT CanHal_ConfigType[CanHal_ConfigType] -.-> pCanIcomCfgs Can_IcomConfigsType[Can_IcomConfigsType] CanHal_ConfigType -.-> pControllerInstances CanHal_ControllerInstanceType[CanHal_ControllerInstanceType] Can_IcomConfigsType -.-> pCanIcomRxMessageConfigs Can_IcomRxMessageConfigsType[Can_IcomRxMessageConfigsType] Can_IcomRxMessageConfigsType -.-> pCanIcomRxMessageSignalConfigs Can_IcomRxMessageSignalType[Can_IcomRxMessageSignalType] </pre>
Data Fields	- const Can_IcomConfigsType* pCanIcomCfgs Pretended Network Configuration Array. - const CanHal_ControllerInstanceType* pControllerInstances Controller Instance Array. - uint32 u32CanObjCount Can Hardware Object Count. - uint8 u8ControllerCount Controller Instance Count. - const uint8 u8NumCanIcomCfgs Pretended Network Configuration count.

2.6.15 Can_RxFiFoTableIdConfigType

Structure	Can_RxFiFoTableIdConfigType
------------------	-----------------------------

Description	Legacy Rx FIFO Configuration.
Diagram	N/A
Data Fields	- const uint32 u32TableFilterMask Mask for Legacy Rx FIFO. - const uint32 u32TableId ID for Legacy Rx FIFO.

2.6.16 CanHal_ControllerInstanceType

Structure	CanHal_ControllerInstanceType
Description	Can Controller hardware Instance Structure.
Diagram	N/A
Data Fields	- const boolean bPnSupported Pretended Network support. - const Can_PCallBackType Can_ErrorFDNotification Pointer to Error interrupt notification function (ESR[ERRFAST_INT]). - const Can_PCallBackType Can_ErrorNotification Pointer to Error interrupt notification function (ESR[ERR_INT]). - const uint32 u32ControlReg CTRL Register value. - const uint32 u32ECUMWakeupSourceId ID source Wakeup of EcuM. - const uint32 u32Options BusOff Sw Recovery, RXFifo En, IDAM Type, Event Trigger Mode TxProcessing/RxProcessing/BusoffProcessing. - const uint8 u8CanIndex Can Index. - const uint8 u8NumberOfMB Number of message Buffers available for FlexCan unit.

2.6.17 CanHal_ControllerStatusType

Structure	CanHal_ControllerStatusType
Description	Can Controller Status Store Structure.
Diagram	N/A
Data Fields	- boolean bTxGuard[32] Can Controller Tx In Processing. - CanHal_IcomStateType eCanIcomState Pretended Network State. - Can_ControllerStateType eControllerState Controller State. - CanHal_InterruptStateType eInterruptMode Interrupt Enabled Or Disabled. - volatile sint8 s8IntDisableLevel

	Interrupt Disable Counter. - uint16 u16MBMapping[CAN_MAXMB_CONFIGURED] Logical MB index Configured in EB. - uint32 u32MBInterruptMask[1] Can Controller Interrupt Mask value, current can only contains mask1. - uint32 u32MBInterruptRxMask[1] Can Controller Rx MB interrupt mask. - uint32 u32MBInterruptTxMask[1] Can Controller Tx MB interrupt mask. - uint8 u8CurrentBaudRateIndex Baudrate Configuration Index in EB. - volatile uint8 u8FirstTxMBIndex First Tx Message Buffer Index. - PduldType u8TxPduld[32] Can Controller Tx Pdu ID.
--	---

2.6.18 CanHw_MbOffsetWithHoh

Structure	CanHw_MbOffsetWithHoh
Description	Store One MB Logical ID in EB and Physical Index in MBs and Offset in Controller.
Diagram	N/A
Data Fields	- uint16 u16HohIndex MB Logical ID in EB. - uint16 u16HwMbIndex MB Index in current Controller. - uint32 u32MbOffset MB Offset in Current Controller.

2.6.19 CanHw_Message

Structure	CanHw_Message
Description	N/A
Diagram	N/A
Data Fields	- uint8 aData[64] - uint32 u32MbMessageId - uint8 u8DataLength

2.7 API Functions

2.7.1 Functions in Can.h

2.7.1.1 void Can_Init(const Can_ConfigType* Config);

Function	void Can_Init(const Can_ConfigType* Config);
Description	This function initializes the module.
Diagram	N/A

Parameters	Parameter	Description
	Config	Pointer to driver configuration.
Returns	N/A	

2.7.1.2 void Can_GetVersionInfo(Std_VersionInfoType* Versioninfo);

Function	void Can_GetVersionInfo(Std_VersionInfoType* Versioninfo);	
Description	This function returns the version information of this module.	
Diagram	N/A	
Parameters	Parameter	Description
	Versioninfo	Pointer to where to store the version information of this module.
Returns	N/A	

2.7.1.3 void Can_Delnit(void);

Function	void Can_Delnit(void);	
Description	This function de-initializes the module.	
Diagram	N/A	
Parameters	N/A	
Returns	N/A	

2.7.1.4 Std_ReturnType Can_SetBaudrate(uint8 Controller, const uint16 BaudRateConfigID);

Function	Std_ReturnType Can_SetBaudrate(uint8 Controller, const uint16 BaudRateConfigID);	
Description	This service shall set the baud rate configuration of the CAN controller. Depending on necessary baud rate modifications the controller might have to reset.	
Diagram	N/A	
Parameters	Parameter	Description
	Controller	CAN controller, whose baud rate shall be set
	Transition	references a baud rate configuration by ID (see CanControllerBaudRateConfigID)
Returns	E_OK: Service request accepted, setting of (new) baud rate started; E_NOT_OK: Service request not accepted	

2.7.1.5 Std_ReturnType Can_SetControllerMode(uint8 Controller, Can_ControllerStateType Transition);

Function	Std_ReturnType Can_SetControllerMode(uint8 Controller, Can_ControllerStateType Transition);	
Description	This function performs software triggered state transitions of the CAN controller State machine.	
Diagram	N/A	
Parameters	Parameter	Description
	Controller	CAN controller for which the status shall be changed
	Transition	Transition value to request new CAN controller state
Returns	E_OK: request accepted; E_NOT_OK: request not accepted, a development error occurred	

2.7.1.6 void Can_DisableControllerInterrupts(uint8 Controller);

Function	void Can_DisableControllerInterrupts(uint8 Controller);	
Description	This function disables all interrupts for this CAN controller.	
Diagram	N/A	
Parameters	Parameter	Description
	Controller	CAN controller for which interrupts shall be disabled.
Returns	N/A	

2.7.1.7 void Can_EnableControllerInterrupts(uint8 Controller);

Function	void Can_EnableControllerInterrupts(uint8 Controller);	
Description	This function enables all allowed interrupts.	
Diagram	N/A	
Parameters	Parameter	Description
	Controller	CAN controller for which interrupts shall be re-enabled
Returns	N/A	

2.7.1.8 Std_ReturnType Can_CheckWakeup(uint8 Controller);

Function	Std_ReturnType Can_CheckWakeup(uint8 Controller);	
Description	This function checks if a wakeup has occurred for the given controller.	
Diagram	N/A	
Parameters	Parameter	Description
	Controller	Controller to be checked for a wakeup.
Returns	E_OK: API call has been accepted; E_NOT_OK: API call has not been accepted	

2.7.1.9 Std_ReturnType Can_GetControllerErrorState(uint8 ControllerId, Can_ErrorStateType* ErrorStatePtr);

Function	Std_ReturnType Can_GetControllerErrorState(uint8 ControllerId, Can_ErrorStateType* ErrorStatePtr);	
Description	This service obtains the error state of the CAN controller.	
Diagram	N/A	
Parameters	Parameter	Description
	ControllerId	Abstracted CanIf ControllerId which is assigned to a CAN controller, which is requested for ErrorState.
	ErrorStatePtr	Pointer to a memory location, where the error state of the CAN controller will be stored.
Returns	E_OK: Error state request has been accepted; E_NOT_OK: Error state request has not been accepted.	

2.7.1.10 Std_ReturnType Can_GetControllerMode(uint8 Controller, Can_ControllerStateType* ControllerModePtr);

Function	Std_ReturnType Can_GetControllerMode(uint8 Controller, Can_ControllerStateType* ControllerModePtr);	
Description	This service reports about the current status of the requested CAN controller.	

Diagram	N/A	
Parameters	Parameter	Description
	Controller	CAN controller for which the status shall be requested.
	ControllerModePtr	Pointer to a memory location, where the current mode of the CAN controller will be stored.
Returns	E_OK: Controller mode request has been accepted; E_NOT_OK: Controller mode request has not been accepted	

2.7.1.11 Std_ReturnType Can_Write(Can_HwHandleType Hth, const Can_PduType* PduInfo);

Function	Std_ReturnType Can_Write(Can_HwHandleType Hth, const Can_PduType* PduInfo);	
Description	This function is called by CanIf to pass a CAN message to CanDrv for transmission.	
Diagram	N/A	
Parameters	Parameter	Description
	Hth	Information which HW-transmit handle shall be used for transmit. Implicitly this is also the information about the controller to use because the Hth numbers are unique inside one hardware unit.
	PduInfo	Pointer to SDU user memory, Data Length and Identifier.
Returns	E_OK: Write command has been accepted; E_NOT_OK: development error occurred CAN_BUSY: No TX hardware buffer available or pre-emptive call of Can_Write that can't be implemented re-entrant (see Can_ReturnType)	

2.7.1.12 void Can_MainFunction_Write(void);

Function	void Can_MainFunction_Write(void);
Description	This function performs the polling of TX confirmation when CAN_TX_PROCESSING is set to POLLING.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.13 void Can_MainFunction_Write_0(void);

Function	void Can_MainFunction_Write_0(void);
Description	This function performs the polling of TX confirmation when CAN_TX_PROCESSING is set to POLLING in process 0(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.14 void Can_MainFunction_Write_1(void);

Function	void Can_MainFunction_Write_1(void);
Description	This function performs the polling of TX confirmation when CAN_TX_PROCESSING is set to POLLING in process 1(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A

Returns	N/A
----------------	-----

2.7.1.15 void Can_MainFunction_Write_2(void);

Function	void Can_MainFunction_Write_2(void);
Description	This function performs the polling of TX confirmation when CAN_TX_PROCESSING is set to POLLING in process 2(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.16 void Can_MainFunction_Write_3(void);

Function	void Can_MainFunction_Write_3(void);
Description	This function performs the polling of TX confirmation when CAN_TX_PROCESSING is set to POLLING in process 3(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.17 void Can_MainFunction_Write_4(void);

Function	void Can_MainFunction_Write_4(void);
Description	This function performs the polling of TX confirmation when CAN_TX_PROCESSING is set to POLLING in process 4(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.18 void Can_MainFunction_Read(void);

Function	void Can_MainFunction_Read(void);
Description	This function performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.19 void Can_MainFunction_Read_0(void);

Function	void Can_MainFunction_Read_0(void);
Description	This function performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING in process 0(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.20 void Can_MainFunction_Read_1(void);

Function	void Can_MainFunction_Read_1(void);
Description	This function performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING in process 1(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.21 void Can_MainFunction_Read_2(void);

Function	void Can_MainFunction_Read_2(void);
Description	This function performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING in process 2(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.22 void Can_MainFunction_Read_3(void);

Function	void Can_MainFunction_Read_3(void);
Description	This function performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING in process 3(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.23 void Can_MainFunction_Read_4(void);

Function	void Can_MainFunction_Read_4(void);
Description	This function performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING in process 4(Different MainRwmainFunction set in EB).
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.24 void Can_MainFunction_BusOff(void);

Function	void Can_MainFunction_BusOff(void);
Description	This function performs the polling of bus-off events that are configured statically as 'to be polled'.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.25 void Can_MainFunction_Wakeup(void);

Function	void Can_MainFunction_Wakeup(void);
-----------------	-------------------------------------

Description	This function performs the polling of wake-up events that are configured statically as 'to be polled'.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.26 void Can_MainFunction_Mode(void);

Function	void Can_MainFunction_Mode(void);
Description	This function performs the polling of CAN controller mode transitions.
Diagram	N/A
Parameters	N/A
Returns	N/A

2.7.1.27 Std_ReturnType Can_SetIcomConfiguration(uint8 u8Controller, IcomConfigIdType u8ConfigurationId);

Function	Std_ReturnType Can_SetIcomConfiguration(uint8 u8Controller, IcomConfigIdType u8ConfigurationId);	
Description	This service shall change the Icom Configuration of a CAN controller to the requested one..	
Diagram	N/A	
Parameters	Parameter	Description
	u8Controller	CAN controller for which the status shall be changed.
	u8ConfigurationId	Requested Configuration
Returns	E_OK: CAN driver succeeded in setting a configuration with a valid Configuration id; E_NOT_OK: CAN driver failed to set a configuration with a valid Configuration id.	

2.8 Hardware Functions

2.8.1 Functions in Can_Hal.h

2.8.1.1 static inline void Can_ReportDetError (uint8 u8ServiceId, uint8 u8ErrorId);

Function	static inline void Can_ReportDetError (uint8 u8ServiceId, uint8 u8ErrorId);	
Description	This function reports an error to the Det module and records the failure in the local structure.	
Parameters	Parameter	Description
	u8ServiceId	The service id of the caller function
	u8ErrorId	The error id to be reported
Returns	void	

2.8.1.2 static inline void Can_ReportRuntimeDetError (uint8 u8ServiceId, uint8 u8ErrorId);

Function	static inline void Can_ReportRuntimeDetError (uint8 u8ServiceId, uint8 u8ErrorId);	
Description	This function reports a running error to the Det module and records the failure in the local structure.	
Parameters	Parameter	Description
	u8ServiceId	The service id of the caller function
	u8ErrorId	The error id to be reported
Returns	void	

2.8.1.3 Can_PCallBackType Can_Hal_GetErrorNotify(uint8 u8ControllerId);

Function	Can_PCallBackType Can_Hal_GetErrorNotify(uint8 u8ControllerId);	
Description	Get Error Notification Function Point.	
Parameters	Parameter	Description
	u8ControllerId	Controller ID
Returns	Function Point	

2.8.1.4 Can_PCallBackType Can_Hal_GetFdErrorNotify(uint8 u8ControllerId);

Function	Can_PCallBackType Can_Hal_GetFdErrorNotify(uint8 u8ControllerId);	
Description	Get Can Fd Error Notification Function Point.	
Parameters	Parameter	Description
	u8ControllerId	Controller ID
Returns	Function Point	

2.8.1.5 void Can_Hal_Init(const Can_ConfigType *pConfigCurrent);

Function	void Can_Hal_Init(const Can_ConfigType *pConfigCurrent);	
Description	Initial All Can Controller and Message Buffer with current configuration.	
Parameters	Parameter	Description
	pConfigCurrent	Current Configuration generated by EB
Returns	N/A	

2.8.1.6 void Can_Hal_DeInit(void);

Function	void Can_Hal_DeInit(void);	
Description	This function de-initializes the module.	
Parameters	N/A	
Returns	N/A	

2.8.1.7 Std_ReturnType Can_Hal_CheckControllerStarted(uint8 u8ControllerId);

Function	Std_ReturnType Can_Hal_CheckControllerStarted(uint8 u8ControllerId);	
Description	Check controller if it is started.	
Parameters	Parameter	Description
	u8Controller	Controller ID
Returns	E_OK means started	

2.8.1.8 Can_ReturnType Can_Hal_SetControllerMode(uint8 u8ControllerId, Can_StateTransitionType Transition, boolean canif_notification_required);

Function	Can_ReturnType Can_Hal_SetControllerMode(uint8 u8ControllerId, Can_StateTransitionType Transition, boolean canif_notification_required);	
Description	Set Controller Mode.	
Parameters	Parameter	Description

	u8ControllerId	Controller Id
	Transition	Transition value to request new CAN controller state
	canif_notification_required	if notify to CanIf
Returns	E_OK means set correctly	

2.8.1.9 void Can_Hal_DisableControllerInterrupts(uint8 u8ControllerId);

Function	void Can_Hal_DisableControllerInterrupts(uint8 u8ControllerId);	
Description	Disable Can Controller Interrupt.	
Parameters	Parameter	Description
	u8ControllerId	Controller id
Returns	N/A	

2.8.1.10 void Can_Hal_EnableControllerInterrupts(uint8 u8ControllerId);

Function	void Can_Hal_EnableControllerInterrupts(uint8 u8ControllerId);	
Description	Enable Can Controller Interrupt.	
Parameters	Parameter	Description
	u8ControllerId	Controller id
Returns	N/A	

2.8.1.11 Std_ReturnType CanHal_CheckWakeup(uint8 u8ControllerId);

Function	Std_ReturnType CanHal_CheckWakeup(uint8 u8ControllerId);	
Description	This function checks if a wakeup has occurred for the given controller.	
Parameters	Parameter	Description
	Controller	Controller to be checked for a wakeup.
Returns	E_OK: API call has been accepted;	
	E_NOT_OK: API call has not been accepted	

2.8.1.12 Std_ReturnType CanHal_GetControllerErrorState(uint8 u8ControllerId, Can_ErrorStateType* ErrorStatePtr);

Function	Std_ReturnType CanHal_GetControllerErrorState(uint8 u8ControllerId, Can_ErrorStateType* ErrorStatePtr);	
Description	This service obtains the error state of the CAN controller.	
Parameters	Parameter	Description
	ControllerId	Abstracted CanIf ControllerId which is assigned to a CAN controller, which is requested for ErrorState.
	ErrorStatePtr	Pointer to a memory location, where the error state of the CAN controller will be stored.
Returns	E_OK: Error state request has been accepted;	
	E_NOT_OK: Error state request has not been accepted.	

2.8.1.13 Std_ReturnType CanHal_GetControllerMode(uint8 u8ControllerId, Can_ControllerStateType* ControllerModePtr);

Function	Std_ReturnType CanHal_GetControllerMode(uint8 u8ControllerId, Can_ControllerStateType* ControllerModePtr);	
Description	This service reports about the current status of the requested CAN controller.	
Parameters	Parameter	Description
	Controller	CAN controller for which the status shall be requested.
	ControllerModePtr	Pointer to a memory location, where the current mode of the CAN controller will be stored.
Returns	E_OK: Controller mode request has been accepted; E_NOT_OK: Controller mode request has not been accepted	

2.8.1.14 Can_ReturnType Can_Hal_Write(Can_HwHandleType hth, const Can_PduType *PduInfo);

Function	Can_ReturnType Can_Hal_Write(Can_HwHandleType hth, const Can_PduType *PduInfo);	
Description	Write Can Message to Hardware object.	
Parameters	Parameter	Description
	hth	Hardware object id
	PduInfo	point to write message
Returns	E_OK means write correctly	

2.8.1.15 Std_ReturnType Can_Hal_SetBaudrate(uint8 u8ControllerId, const uint16 BaudrateConfigID);

Function	Std_ReturnType Can_Hal_SetBaudrate(uint8 u8ControllerId, const uint16 BaudrateConfigID);	
Description	Set can baudrate with controller id and baudrate configuration id in EB.	
Parameters	Parameter	Description
	Controller	controller id
	BaudrateConfigID	baudrate configuration in EB
Returns	E_OK means set correctly	

2.8.1.16 void Can_Hal_ProcessBusOff(const uint8 u8ControllerId);

Function	void Can_Hal_ProcessBusOff(const uint8 u8ControllerId);	
Description	clear bus off error status.	
Parameters	Parameter	Description
	u8ControllerId	Controller id
Returns	N/A	

2.8.1.17 Std_ReturnType Can_Hal_CheckBusOff(uint8 u8ControllerId);

Function	Std_ReturnType Can_Hal_CheckBusOff(uint8 u8ControllerId);	
Description	Check Bus off status.	
Parameters	Parameter	Description
	u8ControllerId	Controller id
Returns	E_OK means bus off appeared	

2.8.1.18 void Can_Hal_BusOffPoll(void);

Function	void Can_Hal_BusOffPoll(void);
Description	Polling All Controller Bus off status.
Parameters	N/A
Returns	N/A

2.8.1.19 void Can_Hal_ProcessTx(const uint8 u8ControllerId, const uint8 mbindex_start, const uint8 mbindex_end);

Function	void Can_Hal_ProcessTx(const uint8 u8ControllerId, const uint8 mbindex_start, const uint8 mbindex_end);	
Description	Process transmit interrupt.	
Parameters	Parameter	Description
	u8ControllerId	Controller id
	mbindex_start	start message buffer index
	mbindex_end	end message buffer index
Returns	N/A	

2.8.1.20 void Can_Hal_MainFunctionWritePoll(void);

Function	void Can_Hal_MainFunctionWritePoll(void);
Description	Polling check if write successfully and indicate to CanIf.
Parameters	N/A
Returns	N/A

2.8.1.21 void Can_Hal_MainFunctionMultipleWritePoll(uint8 writepoll);

Function	void Can_Hal_MainFunctionMultipleWritePoll(uint8 writepoll);	
Description	Multi-Polling for differeent thread with thread id, used in multi-core process(Different MainRwmainFunction set in EB).	
Parameters	Parameter	Description
	writepoll	thread id
Returns	N/A	

2.8.1.22 void Can_Hal_ProcessRx(const uint8 u8ControllerId, const uint8 mbindex_start, const uint8 mbindex_end);

Function	void Can_Hal_ProcessRx(const uint8 u8ControllerId, const uint8 mbindex_start, const uint8 mbindex_end);	
Description	Process Receive Interrupt.	
Parameters	Parameter	Description
	u8ControllerId	Controller id
	mbindex_start	Message Buffer Start index
	mbindex_end	Message Buffer End index
Returns	N/A	

2.8.1.23 void Can_Hal_MainFunctionReadPoll(void);

Function	void Can_Hal_MainFunctionReadPoll(void);
Description	Polling Check If received Message and Indicate to CanIf.
Parameters	N/A
Returns	N/A

2.8.1.24 void Can_Hal_MainFunctionMultipleReadPoll(uint8 readpoll);

Function	void Can_Hal_MainFunctionMultipleReadPoll(uint8 readpoll);	
Description	Multi-Polling for differeent thread with thread id, used in multi-core process (Different MainRwmainFunction set in EB).	
Parameters	Parameter	Description
	readpoll	Thread ID.
Returns	N/A	

2.8.1.25 void Can_Hal_ModePoll(void);

Function	void Can_Hal_ModePoll(void);
Description	Polling Check MCR Mode.
Parameters	N/A
Returns	N/A

2.8.1.26 Std_ReturnType Can_Hal_SetIcomConfiguration(uint8 u8Controller, IcomConfigIdType u8ConfigurationId);

Function	Std_ReturnType Can_Hal_SetIcomConfiguration(uint8 u8Controller, IcomConfigIdType u8ConfigurationId);	
Description	Set Pretended Network configuration.	
Parameters	Parameter	Description
	u8Controller	Controller id
	u8ConfigurationId	Configuration id
Returns	E_OK means set correctly	

2.8.1.27 void Can_Hal_ProcessWakeupPN(const uint8 u8controller);

Function	void Can_Hal_ProcessWakeupPN(const uint8 u8controller);	
Description	Process Can Controller Wakeup.	
Parameters	Parameter	Description
	u8controller	Can Controller Id
Returns	N/A	

2.8.2 Functions in Can_Hw.h**2.8.2.1 void Can_Hw_InitLogicMbOffset(uint16 u16HohIndex, uint16 u16MbIndex, uint32 u32MbOffset);**

Function	void Can_Hw_InitLogicMbOffset(uint16 u16HohIndex, uint16 u16MbIndex, uint32 u32MbOffset);	
Description	Initial MB Logical ID in EB and Physical Index in MBs and Offset in Controller.	
Parameters	Parameter	Description

	u16HohIndex	MB Logical ID in EB
	u16MbIndex	MB Index in current Controller
	u32MbOffset	MB Offset in Current Controller
Returns	N/A	

2.8.2.2 void Can_Hw_Init(void);

Function	void Can_Hw_Init(void);	
Description	Initial Can Hw variables.	
Parameters	N/A	
Returns	N/A	

2.8.2.3 void Can_Hw_InitEmbeddedRam(uint8 u8CanIndex);

Function	void Can_Hw_InitEmbeddedRam(uint8 u8CanIndex);	
Description	Initial Can Controller Embedded Memory, Avoid CAN ECC appearing.	
Parameters	Parameter	Description
	u8CanIndex	CAN Index
Returns	N/A	

2.8.2.4 void Can_Hw_EnablePNMode (uint8 u8CanIndex, boolean bEnable);

Function	void Can_Hw_EnablePNMode (uint8 u8CanIndex, boolean bEnable);	
Description	Disable the pretended network mode.	
Parameters	Parameter	Description
	u8CanIndex	CAN Index
	bEnable	true: enable the mode. false: disable the mode
Returns	N/A	

2.8.2.5 uint32 Can_Hw_GetMCRValue (uint8 u8CanIndex);

Function	uint32 Can_Hw_GetMCRValue (uint8 u8CanIndex);	
Description	GET the MCR register value.	
Parameters	Parameter	Description
	u8CanIndex	CAN Index
Returns	MCR register value	

2.8.2.6 Std_ReturnType Can_Hw_CheckReadyMode (uint8 u8CanIndex);

Function	Std_ReturnType Can_Hw_CheckReadyMode (uint8 u8CanIndex);	
Description	Check if the can module is in disable mode or freeze mode.	
Parameters	Parameter	Description
	u8CanIndex	CAN Index
Returns	E_OK means The can module is not in disable or freeze mode	

2.8.2.7 Std_ReturnType Can_Hw_CheckModuleDisable (uint8 u8CanIndex);

Function	Std_ReturnType Can_Hw_CheckModuleDisable (uint8 u8CanIndex);	
Description	Check if the can module is in disable mode.	
Parameters	Parameter	Description
	u8CanIndex	CAN Index
Returns	E_OK means the can module is in disable mode	

2.8.2.8 Std_ReturnType Can_Hw_CheckFreezeMode (uint8 u8CanIndex);

Function	Std_ReturnType Can_Hw_CheckFreezeMode (uint8 u8CanIndex);	
Description	Check if the can module is in freeze mode.	
Parameters	Parameter	Description
	u8CanIndex,	CAN Index
Returns	E_OK means The can module is in freeze mode	

2.8.2.9 Std_ReturnType Can_Hw_CheckResetDone (uint8 u8CanIndex);

Function	Std_ReturnType Can_Hw_CheckResetDone (uint8 u8CanIndex);	
Description	Check if the can module is in reset done status.	
Parameters	Parameter	Description
	u8CanIndex,	CAN Index
Returns	E_OK means the can module is in reset done status	

2.8.2.10 Std_ReturnType Can_Hw_CheckLowPowerMode (uint8 u8CanIndex);

Function	Std_ReturnType Can_Hw_CheckLowPowerMode (uint8 u8CanIndex);	
Description	Check if the can module is in low power mode.	
Parameters	Parameter	Description
	u8CanIndex,	CAN Index
Returns	E_OK means the can module is in low power mode.	

2.8.2.11 void Can_Hw_EnableLegacyFifo(uint8 u8CanIndex);

Function	void Can_Hw_EnableLegacyFifo(uint8 u8CanIndex);	
Description	Enable Legacy Rx FIFO.	
Parameters	Parameter	Description
	u8CanIndex	CAN Index
Returns	N/A	

2.8.2.12 void Can_Hw_SetLegacyFifoFilterFormat(uint8 u8CanIndex, CanHw_LegacyFifoFilterType eFilterType);

Function	void Can_Hw_SetLegacyFifoFilterFormat(uint8 u8CanIndex, CanHw_LegacyFifoFilterType eFilterType);	
Description	Set Legacy Rx FIFO Filter Format.	
Parameters	Parameter	Description
	u8CanIndex	CAN Index
	eFilterType	

Returns	N/A
----------------	-----

2.8.2.13 uint32 Can_Hw_GetErrorStatus(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetErrorStatus(uint8 u8CanIndex);	
Description	Get Error Status ESR Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	ESR register value	

2.8.2.14 uint32 Can_Hw_GetErrorInterruptFlag(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetErrorInterruptFlag(uint8 u8CanIndex);	
Description	Get Interrupt Flag In ESR Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	Error Interrupt Flag bit without shift	

2.8.2.15 uint32 Can_Hw_GetBusoffInterruptFlag(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetBusoffInterruptFlag(uint8 u8CanIndex);	
Description	Get Bus Off Error Interrupt Flag In ESR Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	Bus Off Interrupt Flag bit without shift	

2.8.2.16 uint32 Can_Hw_GetFdErrorInterruptFlag(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetFdErrorInterruptFlag(uint8 u8CanIndex);	
Description	Get Can FD Error Flag In ESR Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	Can FD Error Flag bit without shift	

2.8.2.17 Std_ReturnType Can_Hw_ChkBusOffStatus(uint8 u8CanIndex);

Function	Std_ReturnType Can_Hw_ChkBusOffStatus(uint8 u8CanIndex);	
Description	Check Bus off status.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	E_OK means Bus off	

2.8.2.18 void Can_Hw_ClrBusoffInterruptFlag(uint8 u8CanIndex);

Function	void Can_Hw_ClrBusoffInterruptFlag(uint8 u8CanIndex);	
Description	Clear Bus Off Error Interrupt Flag in ESR Register.	
Parameters	Parameter	Description

	u8CanIndex	Can Index
Returns	N/A	

2.8.2.19 void Can_Hw_ClrErrorInterruptFlag(uint8 u8CanIndex);

Function	void Can_Hw_ClrErrorInterruptFlag(uint8 u8CanIndex);	
Description	Clear Error Interrupt Flag In ESR Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.20 void Can_Hw_ClrFdErrorInterruptFlag(uint8 u8CanIndex);

Function	void Can_Hw_ClrFdErrorInterruptFlag(uint8 u8CanIndex);	
Description	Clear Can FD Error Interrupt in ESR Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.21 uint32 Can_Hw_GetErrorInterruptMask(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetErrorInterruptMask(uint8 u8CanIndex);	
Description	Get ErrMask bit in CTRL register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	ErrMask bit value without shift	

2.8.2.22 uint32 Can_Hw_GetBusoffInterruptMask(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetBusoffInterruptMask(uint8 u8CanIndex);	
Description	Get Bus Off Interrupt Mask in CTRL Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	BusOff Interrupt Mask bitout with shift	

2.8.2.23 void Can_Hw_SetBusoffInterrupt(uint8 u8CanIndex);

Function	void Can_Hw_SetBusoffInterrupt(uint8 u8CanIndex);	
Description	Attach Bus Off Interrupt Bit Mask.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.24 void Can_Hw_SetErrorInterrupt(uint8 u8CanIndex);

Function	void Can_Hw_SetErrorInterrupt(uint8 u8CanIndex);	
Description	Attach Error Interrupt bit mask.	

Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.25 void Can_Hw_ClrBusoffInterrupt(uint8 u8CanIndex);

Function	void Can_Hw_ClrBusoffInterrupt(uint8 u8CanIndex);	
Description	Remove Bus Off Interrupt Bit mask.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.26 void Can_Hw_ClrErrorInterrupt(uint8 u8CanIndex);

Function	void Can_Hw_ClrErrorInterrupt(uint8 u8CanIndex);	
Description	Remove Error Interrupt Bit Mask.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.27 uint32 Can_Hw_GetFdErrorInterruptMask(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetFdErrorInterruptMask(uint8 u8CanIndex);	
Description	Get Error Interrupt Mask bit In CTRL Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.28 void Can_Hw_SetLegacyFifoNum(uint8 u8CanIndex, uint32 u32Num);

Function	void Can_Hw_SetLegacyFifoNum(uint8 u8CanIndex, uint32 u32Num);	
Description	Set RFFN in CTRL2 Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32Num	RFFN value
Returns	N/A	

2.8.2.29 uint32 Can_Hw_GetIFlag1(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetIFlag1(uint8 u8CanIndex);	
Description	Get IFLAG1 Register value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	IFLAG1 value	

2.8.2.30 Std_ReturnType Can_Hw_GetMBIFlag (uint8 u8CanIndex, uint8 u8MbIndex);

Function	Std_ReturnType Can_Hw_GetMBIFlag (uint8 u8CanIndex, uint8 u8MbIndex);	
Description	Get each MB IFLAG value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8MbIndex	the MB index
Returns	E_OK: the MB flag is set.	

2.8.2.31 uint32 Can_Hw_GetLegacyFifoAvailableIFlag1(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetLegacyFifoAvailableIFlag1(uint8 u8CanIndex);	
Description	Get Legacy Rx FIFO available status.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	0 means no frame received, and others means received at least one frame in Legacy FIFO.	

2.8.2.32 void Can_Hw_SetIFlag1(uint8 u8CanIndex, uint32 u32IFlag);

Function	void Can_Hw_SetIFlag1(uint8 u8CanIndex, uint32 u32IFlag);	
Description	Set IFLAG1 with value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32IFlag	flag value
Returns	N/A	

2.8.2.33 void Can_Hw_SetMBIFlag (uint8 u8CanIndex, uint8 u8MbIndex);

Function	void Can_Hw_SetMBIFlag (uint8 u8CanIndex, uint8 u8MbIndex);	
Description	Set MB IFLAG to W1C.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8MbIndex	the MB index
Returns	void	

2.8.2.34 void Can_Hw_ClrLegacyFifoOverflowIFlag1(uint8 u8CanIndex);

Function	void Can_Hw_ClrLegacyFifoOverflowIFlag1(uint8 u8CanIndex);	
Description	Clear Legacy Rx FIFO overflow status.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.35 void Can_Hw_ClrLegacyFifoWarningIFlag1(uint8 u8CanIndex);

Function	void Can_Hw_ClrLegacyFifoWarningIFlag1(uint8 u8CanIndex);	
Description	Clear Legacy Rx FIFO warning status.	

Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.36 Std_ReturnType Can_Hw_CheckMbFlag(uint8 u8CanIndex, uint16 u16MbIndex);

Function	Std_ReturnType Can_Hw_CheckMbFlag(uint8 u8CanIndex, uint16 u16MbIndex);	
Description	Check Message buffer status.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16MbIndex	Message Buffer Index
Returns	E_OK means Flag active	

2.8.2.37 uint32 Can_Hw_GetIMask1(uint8 u8CanIndex);

Function	uint32 Can_Hw_GetIMask1(uint8 u8CanIndex);	
Description	Get IMASK1 Register value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	IMASK1 value	

2.8.2.38 void Can_Hw_SetIMask1(uint8 u8CanIndex, uint32 u32IMask);

Function	void Can_Hw_SetIMask1(uint8 u8CanIndex, uint32 u32IMask);	
Description	Set IMASK1 Register value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32IMask	mask value
Returns	N/A	

2.8.2.39 void Can_Hw_SetGlobalMask(uint8 u8CanIndex, uint32 u32IMask);

Function	void Can_Hw_SetGlobalMask(uint8 u8CanIndex, uint32 u32IMask);	
Description	Set Global Message Buffer ID Mask except MB14 and MB15.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32IMask	mask value
Returns	N/A	

2.8.2.40 void Can_Hw_SetRx14Mask(uint8 u8CanIndex, uint32 u32IMask);

Function	void Can_Hw_SetRx14Mask(uint8 u8CanIndex, uint32 u32IMask);	
Description	Set MB14 ID Mask.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32IMask	mask value

Returns	N/A
----------------	-----

2.8.2.41 void Can_Hw_SetRx15Mask(uint8 u8CanIndex, uint32 u32IMask);

Function	void Can_Hw_SetRx15Mask(uint8 u8CanIndex, uint32 u32IMask);	
Description	Set MB15 ID Mask.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32IMask	mask value
Returns	N/A	

2.8.2.42 void Can_Hw_InitRxIndividualMask(uint8 u8CanIndex, uint16 u16MbIndex, uint32 u32Mask);

Function	void Can_Hw_InitRxIndividualMask(uint8 u8CanIndex, uint16 u16MbIndex, uint32 u32Mask);	
Description	Set Individual MB ID Mask for each MB.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16MbIndex	MB Index
	u32Mask	mask value
Returns	N/A	

2.8.2.43 void Can_Hw_WriteFilterMask(uint8 u8CanIndex, uint8 u8RxFifoTableIdIndex, uint32 u32TableFilterMask);

Function	void Can_Hw_WriteFilterMask(uint8 u8CanIndex, uint8 u8RxFifoTableIdIndex, uint32 u32TableFilterMask);	
Description	Set Individual ID Mask for Rx Legacy FIFO.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8RxFifoTableIdIndex	Rx FIFO ID index in Table
	u32TableFilterMask	mask value
Returns	N/A	

2.8.2.44 void Can_Hw_SetGlobalRxMask(uint8 u8CanIndex, uint32 u32Mask);

Function	void Can_Hw_SetGlobalRxMask(uint8 u8CanIndex, uint32 u32Mask);	
Description	Set Legacy Rx FIFO Global Mask.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32Mask	Mask value
Returns	N/A	

2.8.2.45 void Can_Hw_InitReceiveMBs(uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32MsgId);

Function	void Can_Hw_InitReceiveMBs(uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32MsgId);	
Description	Start Receive Message with Special ID in Special MB.	
Parameters	Parameter	Description

	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
	u32MsgId	CAN ID
Returns	N/A	

2.8.2.46 void Can_Hw_EnableTransmit(uint8 u8CanIndex, uint16 u16HohIndex);

Function	void Can_Hw_EnableTransmit(uint8 u8CanIndex, uint16 u16HohIndex);	
Description	Start Transmit Message in Special MB.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
Returns	N/A	

2.8.2.47 void Can_Hw_SetDlcAndTransmitWithConfig(uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32MbConfig);

Function	void Can_Hw_SetDlcAndTransmitWithConfig(uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32MbConfig);	
Description	Set MB DLC and Transmit with configuration value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
	u32MbConfig	Configuration value for MB 1th word
Returns	N/A	

2.8.2.48 void Can_Hw_SetTxId(uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32MbMessageId);

Function	void Can_Hw_SetTxId(uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32MbMessageId);	
Description	Set Transmit ID in Special MB with Local Priority.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
	u32MsgId	CAN ID
Returns	N/A	

2.8.2.49 Std_ReturnType Can_Hw_CheckTxMbEmpty(uint8 u8CanIndex, uint16 u16HohIndex);

Function	Std_ReturnType Can_Hw_CheckTxMbEmpty(uint8 u8CanIndex, uint16 u16HohIndex);	
Description	Check Transmit MB If it is empty.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
Returns	E_OK means Tx MB is empty	

2.8.2.50 Std_ReturnType Can_Hw_CheckRxMbBusy(uint8 u8CanIndex, uint16 u16HohIndex);

Function	Std_ReturnType Can_Hw_CheckRxMbBusy(uint8 u8CanIndex, uint16 u16HohIndex);	
Description	Check Rx MB if it is busy.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
Returns	E_OK means it is busy	

2.8.2.51 Std_ReturnType Can_Hw_WaitMbOutBusy(uint8 u8CanIndex, uint16 u16HohIndex);

Function	Std_ReturnType Can_Hw_WaitMbOutBusy(uint8 u8CanIndex, uint16 u16HohIndex);	
Description	Wait MB exit Rx Busy status.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
Returns	N/A	

2.8.2.52 uint8 Can_Hw_GetMbDataMaxMin (uint8 u8Offset, uint8 bMax);

Function	uint8 Can_Hw_GetMbDataMaxMin (uint8 u8Offset, uint8 bMax);	
Description	Get Mailbox maximum or minimum data length.	
Parameters	Parameter	Description
	u8Offset	Mailbox type offset
	bMax	if it is maximum
Returns	data length	

2.8.2.53 Std_ReturnType Can_Hw_GetMbMsg(uint8 u8CanIndex, uint16 u16HohIndex, CanHw_Message *pRxData, uint32 u32TempFlag);

Function	Std_ReturnType Can_Hw_GetMbMsg(uint8 u8CanIndex, uint16 u16HohIndex, CanHw_Message *pRxData, uint32 u32TempFlag);	
Description	Get Can MB Message.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
	pRxData	Point to data structure
	u32TempFlag	value to write to IFlag1
Returns	E_OK means Read Correctly	

2.8.2.54 void Can_Hw_GetMbData(uint8 u8CanIndex, uint16 u16HohIndex, uint8* pData, uint8 u8WordLength);

Function	void Can_Hw_GetMbData(uint8 u8CanIndex, uint16 u16HohIndex, uint8* pData, uint8 u8WordLength);	
Description	Get Data From MB.	
Parameters	Parameter	Description

	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
	pData	Point to data buffer
	u8WordLength	Get data word length, 1 word = 4 bytes
Returns	N/A	

2.8.2.55 void Can_Hw_GetFreeTimer (uint8 u8CanIndex);

Function	void Can_Hw_GetFreeTimer (uint8 u8CanIndex);	
Description	Read Rx MB Timer to release MB.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.56 void Can_Hw_FillMbData(uint8 u8CanIndex, uint16 u16HohIndex, uint8* pData, uint8 u8DataStart, uint8 u8DataLength);

Function	void Can_Hw_FillMbData(uint8 u8CanIndex, uint16 u16HohIndex, uint8* pData, uint8 u8DataStart, uint8 u8DataLength);	
Description	Fill Message Buffer Data by bytes.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
	pData	Point to Data Buffer
	u8DataStart	Data Start Index
	u8DataLength	Data Total Length
Returns	N/A	

2.8.2.57 void Can_Hw_FillMbDataWithPad(uint8 u8CanIndex, uint16 u16HohIndex, uint8 u8PadData, uint8 u8DataStart, uint8 u8DataLength);

Function	void Can_Hw_FillMbDataWithPad(uint8 u8CanIndex, uint16 u16HohIndex, uint8 u8PadData, uint8 u8DataStart, uint8 u8DataLength);	
Description	Fill Message Buffer Data by bytes.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	Logical MB Index configured in EB
	u8PadData	Pad data to fill
	u8DataStart	Data Start Index
	u8DataLength	Data Total Length
Returns	N/A	

2.8.2.58 void Can_Hw_WriteldToLegacyFifoTable(uint8 u8CanIndex, uint8 u8RxFifoTableIdIndex, uint32 u32TableId);

Function	void Can_Hw_WriteldToLegacyFifoTable(uint8 u8CanIndex, uint8 u8RxFifoTableIdIndex, uint32 u32TableId);	
Description	Write Received ID in Rx FIFO Table.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8RxFifoTableIdIndex	Rx FIFO index in Table
	u32TableId	CAN ID
Returns	N/A	

2.8.2.59 void Can_Hw_DisableEcc (uint8 u8CanIndex);

Function	void Can_Hw_DisableEcc (uint8 u8CanIndex);	
Description	Disables completely the memory detection and correction mechanism.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.60 uint32 Can_Hw_GetMBInterruptStatus (uint8 u8CanIndex);

Function	uint32 Can_Hw_GetMBInterruptStatus (uint8 u8CanIndex);	
Description	Get MB interrupt status value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	interrupt status value	

2.8.2.61 uint32 Can_Hw_GetERFSR (uint8 u8CanIndex);

Function	uint32 Can_Hw_GetERFSR (uint8 u8CanIndex);	
Description	Get ERFSR with value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	ERFSR register value	

2.8.2.62 void Can_Hw_SetERFSR (uint8 u8CanIndex, uint32 u32Status);

Function	void Can_Hw_SetERFSR (uint8 u8CanIndex, uint32 u32Status);	
Description	Set ERFSR with value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32Status	Status value
Returns	N/A	

2.8.2.63 uint32 Can_Hw_GetERFIER (uint8 u8CanIndex);

Function	uint32 Can_Hw_GetERFIER (uint8 u8CanIndex);
-----------------	---

Description	Get ERFIER with value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	ERFIER register value	

2.8.2.64 void Can_Hw_EnableEnhancedFifo (uint8 u8CanIndex);

Function	void Can_Hw_EnableEnhancedFifo (uint8 u8CanIndex);	
Description	Enable Enhanced Rx FIFO.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.65 void Can_Hw_SetEnhancedFifoFilterNumber (uint8 u8CanIndex, uint8 u8NEXIF, uint8 u8NFE);

Function	void Can_Hw_SetEnhancedFifoFilterNumber (uint8 u8CanIndex, uint8 u8NEXIF, uint8 u8NFE);	
Description		
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8NEXIF	NEXIF value
	u8NFE	NFE value
Returns	N/A	

2.8.2.66 void Can_Hw_SetEnhancedFifoWatermark (uint8 u8CanIndex, uint8 u8Watermark);

Function	void Can_Hw_SetEnhancedFifoWatermark (uint8 u8CanIndex, uint8 u8Watermark);	
Description	Set the enhanced fifo watermark.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8RxFifoTableIdIndex	watermark value
Returns	N/A	

2.8.2.67 void Can_Hw_WriteEnhancedFifoFilterTable (uint8 u8CanIndex, uint8 u8TableIndex, uint32 u32Filter);

Function	void Can_Hw_WriteEnhancedFifoFilterTable (uint8 u8CanIndex, uint8 u8TableIndex, uint32 u32Filter);	
Description	Write filter table in Rx FIFO filter register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8TableIndex	Rx FIFO index in Table
	u32Filter	Filter value
Returns	N/A	

2.8.2.68 Std_ReturnType Can_Hw_GetEnhancedFifoMsg (uint8 u8CanIndex, CanHw_Message *pRxData);

Function	Std_ReturnType Can_Hw_GetEnhancedFifoMsg (uint8 u8CanIndex, CanHw_Message *pRxData);	
Description	Get Can enhanced fifo Message.	

Parameters	Parameter	Description
	u8CanIndex	Can Index
	pRxData	Point to data structure
Returns	E_OK means Read Correctly	

2.8.2.69 void Can_Hw_EnableEnhancedFifoInterrupt (uint8 u8CanIndex, uint32 u32Mask);

Function	void Can_Hw_EnableEnhancedFifoInterrupt (uint8 u8CanIndex, uint32 u32Mask);	
Description	Enable enhanced Rx FIFO interrupt.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32Mask	Interrupt mask
Returns	N/A	

2.8.2.70 void Can_Hw_DisableEnhancedFifoInterrupt (uint8 u8CanIndex, uint32 u32Mask);

Function	void Can_Hw_DisableEnhancedFifoInterrupt (uint8 u8CanIndex, uint32 u32Mask);	
Description	Disable enhanced Rx FIFO interrupt.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32Mask	Interrupt mask
Returns	N/A	

2.8.2.71 uint32 Can_Hw_GetEnhancedFifoInterruptStatus (uint8 u8CanIndex);

Function	uint32 Can_Hw_GetEnhancedFifoInterruptStatus (uint8 u8CanIndex);	
Description	Get the enhanced fifo Interrupt status.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	Enhanced fifo interrupt status	

2.8.2.72 void Can_Hw_EnableModule (uint8 u8CanIndex);

Function	void Can_Hw_EnableModule (uint8 u8CanIndex);	
Description	Enable CAN module.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.73 void Can_Hw_DisableModule (uint8 u8CanIndex);

Function	void Can_Hw_DisableModule (uint8 u8CanIndex);	
Description	Disable CAN module.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.74 void Can_Hw_ClearErrorStatus (uint8 u8CanIndex, uint32 u32value);

Function	void Can_Hw_ClearErrorStatus (uint8 u8CanIndex, uint32 u32value);	
Description	Clear ESR Register.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32value	Register value
Returns	N/A	

2.8.2.75 void Can_Hw_ClearFifoOperation (uint8 u8CanIndex);

Function	void Can_Hw_ClearFifoOperation (uint8 u8CanIndex);	
Description	Trigger the clear FIFO operation	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.76 void Can_Hw_ClrFreezeRequest (uint8 u8CanIndex);

Function	void Can_Hw_ClrFreezeRequest (uint8 u8CanIndex);	
Description	Clear freeze request.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.77 void Can_Hw_SetFreezeRequest (uint8 u8CanIndex);

Function	void Can_Hw_SetFreezeRequest (uint8 u8CanIndex);	
Description	Set freeze request	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.78 void Can_Hw_DisableSelfReceptionMode (uint8 u8CanIndex);

Function	void Can_Hw_DisableSelfReceptionMode (uint8 u8CanIndex);	
Description	Disable Self Reception mode.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.79 void Can_Hw_EnableSelfReceptionMode (uint8 u8CanIndex);

Function	void Can_Hw_EnableSelfReceptionMode (uint8 u8CanIndex);	
Description	Enable Self Reception mode.	
Parameters	Parameter	Description

	u8CanIndex	Can Index
Returns	N/A	

2.8.2.80 void Can_Hw_RequestSoftReset (uint8 u8CanIndex);

Function	void Can_Hw_RequestSoftReset (uint8 u8CanIndex);	
Description	request soft reset.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.81 void Can_Hw_EnableIndividualMask (uint8 u8CanIndex);

Function	void Can_Hw_EnableIndividualMask (uint8 u8CanIndex);	
Description	Enable Individual Rx Masking.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.82 void Can_Hw_EnableLocalPriority (uint8 u8CanIndex);

Function	void Can_Hw_EnableLocalPriority (uint8 u8CanIndex);	
Description	Enable Local Priority.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.83 void Can_Hw_AbortTransmit (uint8 u8CanIndex);

Function	void Can_Hw_AbortTransmit (uint8 u8CanIndex);	
Description	Abort can transmit.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.84 void Can_Hw_SetMAXMBNumber (uint8 u8CanIndex, uint8 u8MBNumber);

Function	void Can_Hw_SetMAXMBNumber (uint8 u8CanIndex, uint8 u8MBNumber);	
Description	Set MAX MB number.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8MBNumber	The number of the used MB
Returns	N/A	

2.8.2.85 void Can_Hw_ClrMB (uint8 u8CanIndex, uint8 u8MBIndex);

Function	void Can_Hw_ClrMB (uint8 u8CanIndex, uint8 u8MBIndex);	
-----------------	--	--

Description	clear MB ram value to all zero.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8MBIndex	The Mailbox index
Returns	N/A	

2.8.2.86 void Can_Hw_EnableWRMFRZ (uint8 u8CanIndex);

Function	void Can_Hw_EnableWRMFRZ (uint8 u8CanIndex);	
Description	Enable unrestricted write access to FLEXCAN memory.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.87 void Can_Hw_SetTASD (uint8 u8CanIndex, uint32 u32Value);

Function	void Can_Hw_SetTASD (uint8 u8CanIndex, uint32 u32Value);	
Description	Set Tx Arbitration Start Delay.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32Value	The delay value
Returns	N/A	

2.8.2.88 uint32 Can_Hw_GetMBConfig (uint8 u8CanIndex, uint16 u16HohIndex);

Function	uint32 Can_Hw_GetMBConfig (uint8 u8CanIndex, uint16 u16HohIndex);	
Description	Get MB configure.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.89 void Can_Hw_SelectBusClock (uint8 u8CanIndex, boolean bBusClockEn);

Function	void Can_Hw_SelectBusClock (uint8 u8CanIndex, boolean bBusClockEn);	
Description	Select can engine clock source.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	bBusClockEn	true: select bus clock as can engine clock source
Returns	N/A	

2.8.2.90 void Can_Hw_SetCBTValue (uint8 u8CanIndex, uint32 u32value);

Function	void Can_Hw_SetCBTValue (uint8 u8CanIndex, uint32 u32value);	
Description	Set CBT register value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index

	u32value	Register value
Returns	N/A	

2.8.2.91 void Can_Hw_SetCtrl (uint8 u8CanIndex, uint32 u32Value);

Function	void Can_Hw_SetCtrl (uint8 u8CanIndex, uint32 u32Value);	
Description	Set CTRL Register Value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32value	Register value
Returns	N/A	

2.8.2.92 void Can_Hw_SetDefaultRegs (uint8 u8CanIndex);

Function	void Can_Hw_SetDefaultRegs (uint8 u8CanIndex);	
Description	reset the register to default value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.93 void Can_Hw_SetMBConfig (uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32Value);

Function	void Can_Hw_SetMBConfig (uint8 u8CanIndex, uint16 u16HohIndex, uint32 u32Value);	
Description	Set MB configure.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u16HohIndex	the MB index
	u32Value	the register value
Returns	N/A	

2.8.2.94 void Can_Hw_SWRecoverFromBOFF (uint8 u8CanIndex);

Function	void Can_Hw_SWRecoverFromBOFF (uint8 u8CanIndex);	
Description	software recovers from bus off state.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.95 void Can_Hw_EnableFDMode (uint8 u8CanIndex);

Function	void Can_Hw_EnableFDMode (uint8 u8CanIndex);	
Description	Enable CANFD mode.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.96 void Can_Hw_DisableBRS (uint8 u8CanIndex);

Function	void Can_Hw_DisableBRS (uint8 u8CanIndex);	
Description	Disable Bit Rate Switch.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.97 void Can_Hw_DisableTDC (uint8 u8CanIndex);

Function	void Can_Hw_DisableTDC (uint8 u8CanIndex)	
Description	Disable Transceiver Delay Compensation.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.98 void Can_Hw_EnableBRS (uint8 u8CanIndex);

Function	void Can_Hw_EnableBRS (uint8 u8CanIndex);	
Description	Enable Bit Rate Switch.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u8RxFifoTableIdIndex	Rx FIFO index in Table
Returns	N/A	

2.8.2.99 void Can_Hw_EnableTDC (uint8 u8CanIndex);

Function	void Can_Hw_EnableTDC (uint8 u8CanIndex);	
Description	Enable Transceiver Delay Compensation.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
Returns	N/A	

2.8.2.100 void Can_Hw_SetFDCBTValue (uint8 u8CanIndex, uint32 u32value)

Function	void Can_Hw_SetFDCBTValue (uint8 u8CanIndex, uint32 u32value)	
Description	Set FDCBT register value.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32value	Register value
Returns	N/A	

2.8.2.101 void Can_Hw_SetFDCTRL2 (uint8 u8CanIndex, uint32 u32Value);

Function	void Can_Hw_SetFDCTRL2 (uint8 u8CanIndex, uint32 u32Value);	
Description	Set CTRL2 register for CANFD.	
Parameters	Parameter	Description

	u8CanIndex	Can Index
	u32value	Register value
Returns	N/A	

2.8.2.102 void Can_Hw_SetTDCValue (uint8 u8CanIndex, uint32 u32Value)

Function	void Can_Hw_SetTDCValue (uint8 u8CanIndex, uint32 u32Value);	
Description	Set Transceiver Delay Compensation Value in CANFD mode.	
Parameters	Parameter	Description
	u8CanIndex	Can Index
	u32Value	The Transceiver Delay Compensation Value
Returns	N/A	

2.8.3 Functions in Can_Irq.c

2.8.3.1 ISR(Can_IsrCAN0_All)

Function	ISR(Can_IsrCAN0_All)
Description	Can0 interrupt handler entry
Parameters	N/A
Returns	N/A

2.8.3.2 ISR(Can_IsrCAN1_All)

Function	ISR(Can_IsrCAN1_All)
Description	Can1 interrupt handler entry
Parameters	N/A
Returns	N/A

2.8.3.3 ISR(Can_IsrCAN2_All)

Function	ISR(Can_IsrCAN2_All)
Description	Can2 interrupt handler entry
Parameters	N/A
Returns	N/A

2.8.3.4 ISR(Can_IsrCAN3_All)

Function	ISR(Can_IsrCAN3_All)
Description	Can3 interrupt handler entry
Parameters	N/A
Returns	N/A

2.8.3.5 ISR(Can_IsrCAN4_All)

Function	ISR(Can_IsrCAN4_All)
Description	Can4 interrupt handler entry

Parameters	N/A
Returns	N/A

2.8.3.6 ISR(Can_IsrCAN5_All)

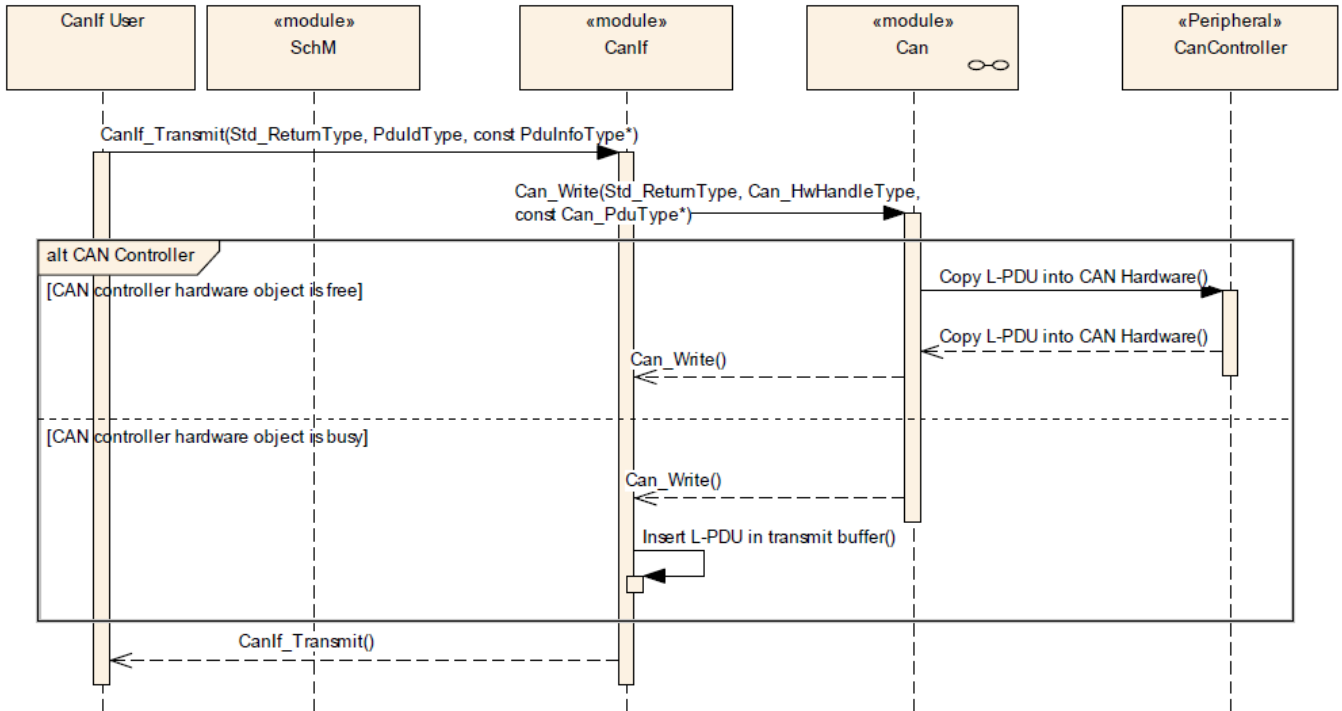
Function	ISR(Can_IsrCAN5_All)
Description	Can5 interrupt handler entry
Parameters	N/A
Returns	N/A

2.9 API Sequence Diagram

The following sequence diagrams show the interactions between CanIf and CanDrv.

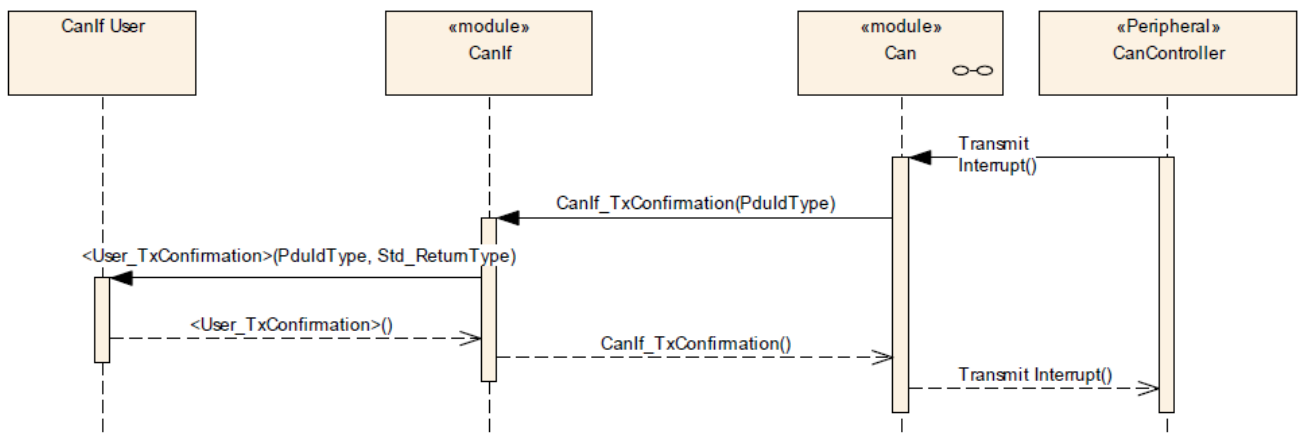
2.9.1 Transmit Request (Single CAN Driver)

Activity	Description
Transmission request	The upper layer initiates a transmit request via the service CanIf_Transmit(). The parameter CanTxPduId identifies the requested L-SDU. The service performs following steps: - validation of the input parameter - definition of the CAN Controller to be used The second parameter *PduInfoPtr is a pointer on the structure with transmit L-SDU related data such as SduLength and *SduDataPtr.
Start transmission	CanIf_Transmit() requests a transmission and calls the CanDrv service Can_Write() with corresponding processing of the HTH.
Hardware request	Can_Write() writes all L-PDU data in the CAN Hardware (if it is free) and sets the hardware request for transmission.
E_OK from Can_Write service	Can_Write() returns E_OK to CanIf_Transmit().
CAN_BUSY from Can_Write service	If CanDrv detects, there are no free hardware objects available, it returns CAN_BUSY to CanIf.
Copying into the buffer	The L-PDU of the rejected transmit request will be inserted in the transmit buffer of CanIf until the next transmit confirmation.
E_OK from CanIf	CanIf_Transmit() returns E_OK to the upper layer.



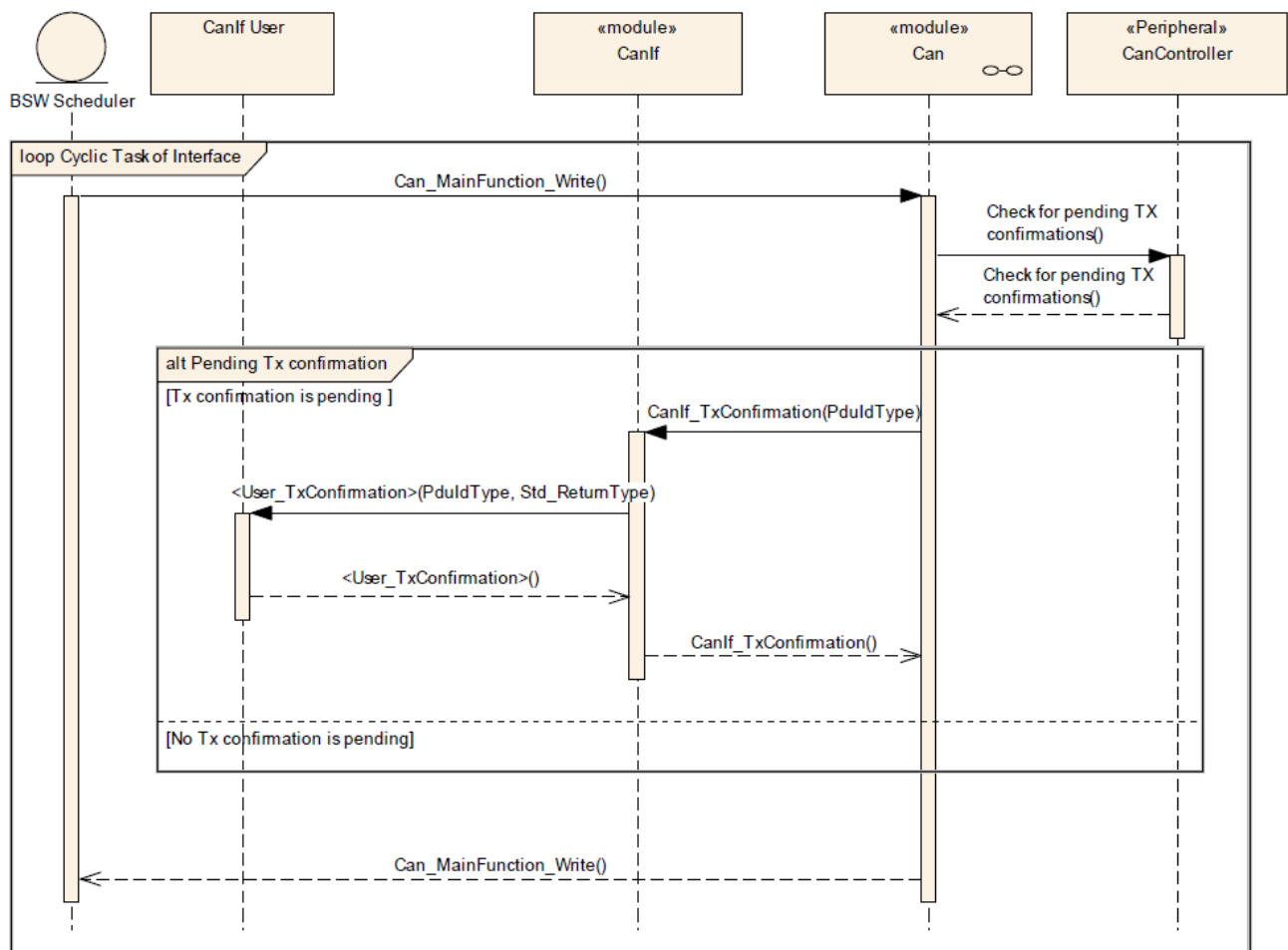
2.9.2 Transmit Confirmation (Interrupt Mode)

Activity	Description
Transmit interrupt	The acknowledged CAN frame signals a successful transmission to the receiving CAN Controller and triggers the transmit interrupt.
Confirmation to CanIf	CanDrv calls the service CanIf_TxConfirmation(). The parameter CanTxPduId specifies the L-PDU previously sent by Can_Write(). CanDrv must store the all in HTHs pending L-PDU Ids in an array organized per HTH to avoid new search of the L-PDU ID for call of CanIf_TxConfirmation().
Confirmation to upper layer	Calling of the corresponding upper layer confirmation service <User_TxConfirmation>(id, E_OK). It signals a successful L-SDU transmission to the upper layer.



2.9.3 Transmit Confirmation (Polling Mode)

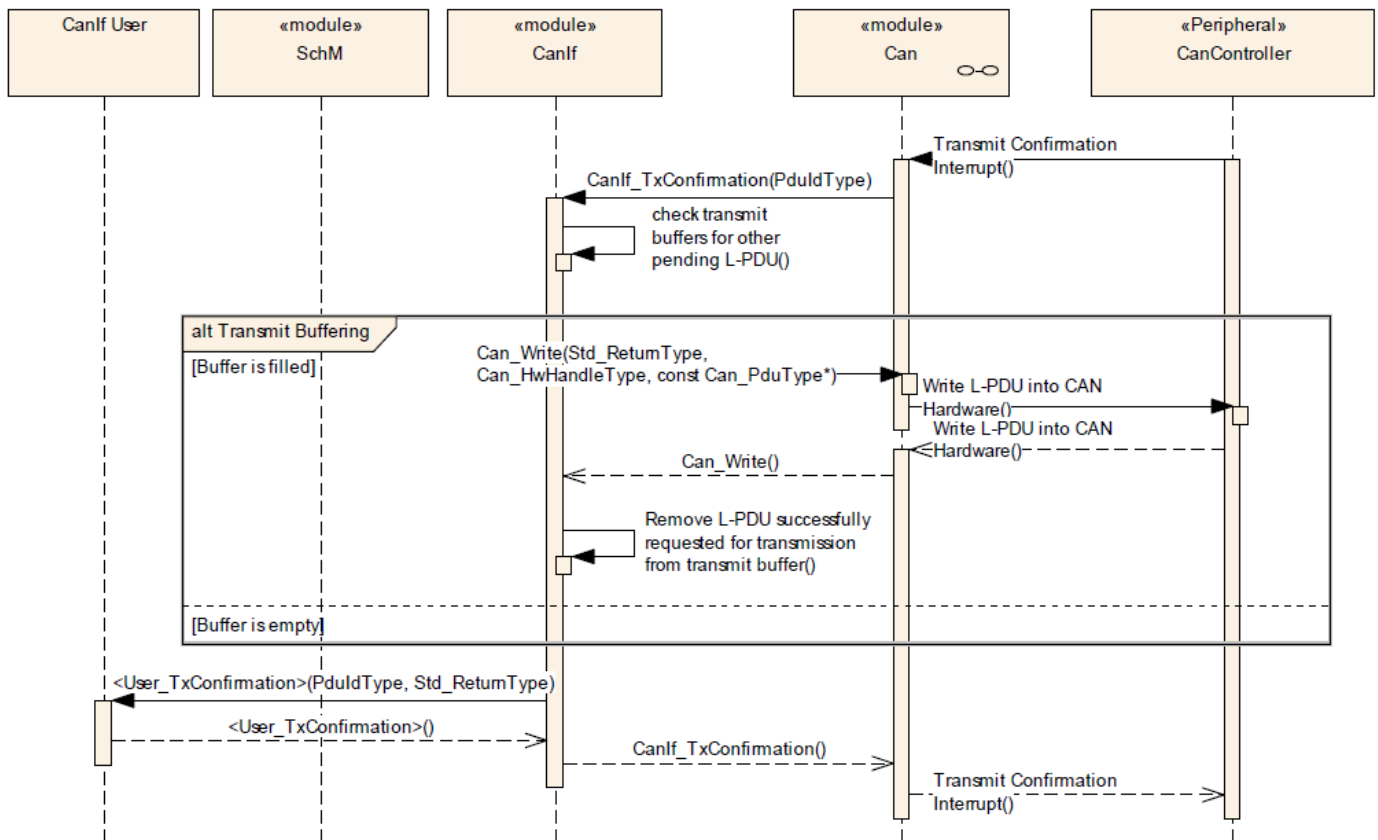
Activity	Description
Cyclic Task CanDrv	The service Can_MainFunction_Write() is called by the BSW Scheduler.
Check for pending transmit confirmations	Can_MainFunction_Write() checks the underlying CAN Controller(s) about pending transmit confirmations of previously succeeded transmit events.
Transmit Confirmation	The acknowledged CAN frame signals a successful transmission to the sending CAN Controller.
Confirmation to CanIf	CanDrv calls the service CanIf_TxConfirmation(). The parameter CanTxPduId specifies the L-PDU previously sent by Can_Write(). CanDrv must store the all in HTHs pending L-PDU Ids in an array organized per HTH to avoid new search of the L-PDU ID for call of CanIf_TxConfirmation().
Confirmation to upper layer	Calling of the corresponding upper layer confirmation service <User_TxConfirmation>(id, E_OK). It signals a successful L-SDU transmission to the upper layer.



2.9.4 Transmit Confirmation (with Buffering)

Activity	Description
Transmit interrupt	Acknowledged CAN frame signals successful transmission to receiving CAN Controller and triggers transmit interrupt.

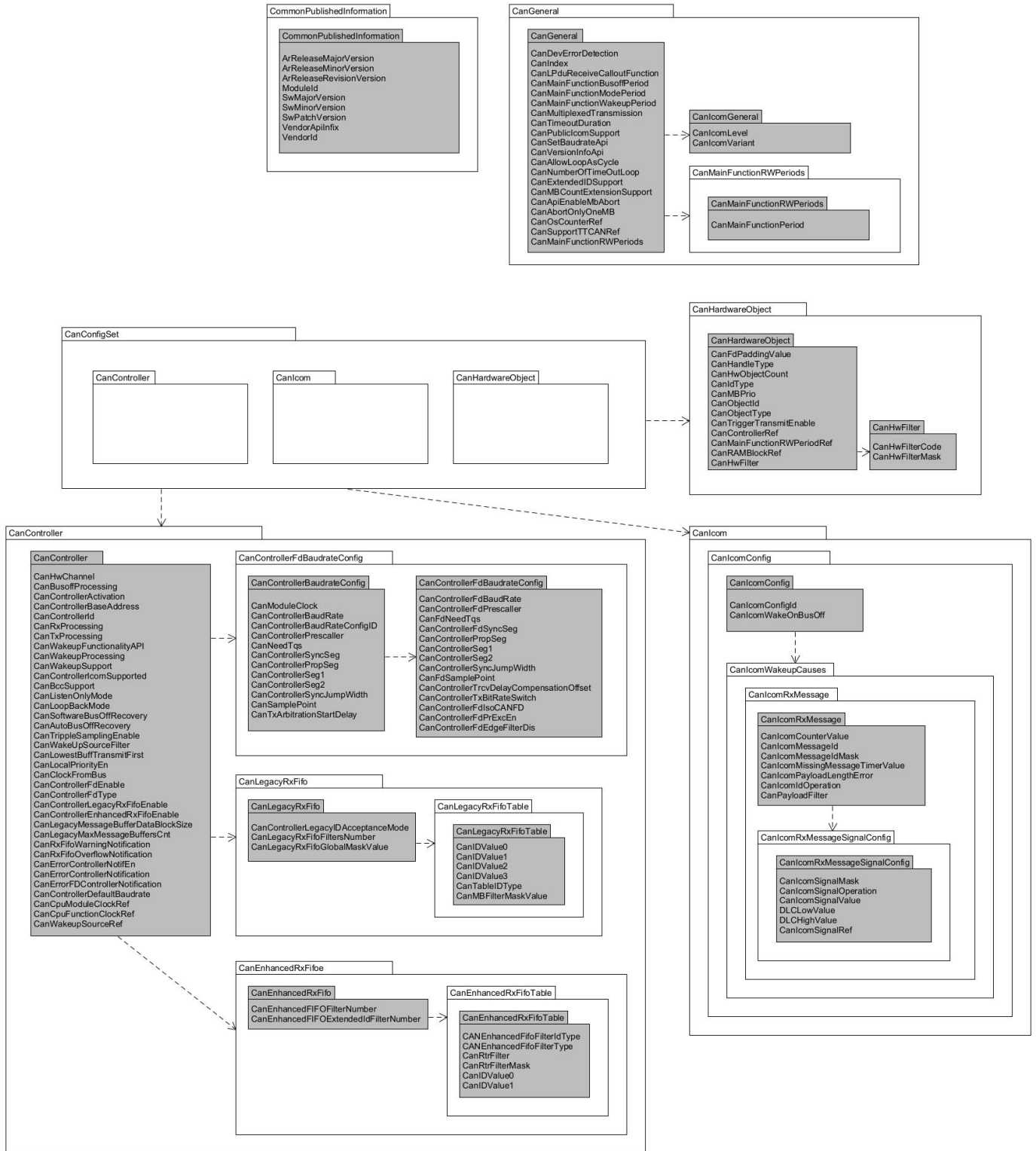
Confirmation to CanIf	CanDrv calls service CanIf_TxConfirmation(). Parameter CanTxPduId specifies the L-PDU previously transmitted by Can_Write(). CanDrv must store the all in HTHs pending L-PDU Ids in an array organized per HTH to avoid new search of the L-PDU ID for call of CanIf_TxConfirmation().
Check of transmit buffers	The transmit buffers of CanIf checked, whether a pending L-PDU is stored or not.
Transmit request passed to CanDrv	In case of pending L-PDUs in the transmit buffers the highest priority order the latest L-PDU is requested for transmission by Can_Write(). It signals a successful L-PDU transmission to the upper layer. Thus Can_Write() can be called re-entrant.
Remove transmitted L-PDU from transmit buffers	The L-PDU pending for transmission is removed from the transmission buffers by CanIf.
Confirmation to the upper layer	Calling of the corresponding upper layer confirmation service <User_TxConfirmation>(id, E_OK). It signals a successful L-SDU transmission to the upper layer.



Chapter 3 Tresos Configuration Items

3.1 Container Inclusion Relation





3.2 Containers and variables

3.2.1 CanGeneral

Container	CanGeneral
Description	This container holds the parameters related to each CAN Driver Unit.
Screenshot	

Properties	Property	Value
	Type	IDENTIFIABLE

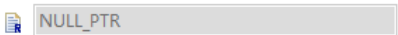
3.2.1.1 Development Error Detection

Variable	CanDevErrorDetection	
Description	Switches the development error detection and notification on or off. - true: enabled (on). - false: disabled (off).	
Screenshot	Development Error Detection 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

3.2.1.2 Can Driver Index

Variable	CanIndex	
Description	Specifies the InstanceId of this module instance. If only one instance is present it shall have the Id 0.	
Screenshot	Can Driver Index (0 -> 255) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

3.2.1.3 CanLPduReceiveCalloutFunction

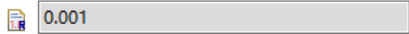
Variable	CanLPduReceiveCalloutFunction	
Description	This parameter defines the existence and the name of a callout function that is called after a successful reception of a received CAN Rx L-PDU. If this parameter is omitted no callout shall take place.	
Screenshot	 CanLPduReceiveCalloutFunction 	
Properties	Property	Value
	Type	FUNCTION-NAME
	SymbolicName	false
	Default	NULL_PTR
	Origin	AUTOSAR_ECUC

3.2.1.4 Can Main Function Busoff Period (seconds)

Variable	CanMainFunctionBusoffPeriod	
Description	ECUC_Can_00355: This parameter describes the period for cyclic call to Can_MainFunction_Busoff. Unit is seconds.	
Screenshot	 Can Main Function Busoff Period (seconds) (0.001 -> 65.535) 	

Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Default	0.001
	Origin	AUTOSAR_ECUC

3.2.1.5 Can Main Function Mode Period (seconds)

Variable	CanMainFunctionModePeriod	
Description	This parameter describes the period for cyclic call to Can_MainFunction_Mode. Unit in seconds.	
Screenshot	Can Main Function Mode Period (seconds) (0.001 -> 65.535) 	
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Default	0.001
	Origin	AUTOSAR_ECUC

3.2.1.6 Can Main Function Wakeup Period (seconds)

Variable	CanMainFunctionWakeupPeriod	
Description	This parameter describes the period for cyclic call to Can_MainFunction_Wakeup. Unit is seconds. when Can->CanConfigSet->CanController->CanWakeupSupport is 'true', editable. wakeup is not supported in FC7300	
Screenshot	 Can Main Function Wakeup Period (seconds) (0.001 -> 65.535) 	
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Default	0.0010
	Origin	AUTOSAR_ECUC

3.2.1.7 Can Multiplexed Transmission

Variable	CanMultiplexedTransmission	
Description	Specifies if Multiplexed Transmission shall be supported: ON or OFF. Multiplex transmission means to search for a free MB, that has the same ObjectId with the one transmitted to Can_Write, if current Hth MB is busy. dependency: CAN Hardware Unit supports multiplexed transmission	
Screenshot	Can Multiplexed Transmission 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	AUTOSAR_ECUC

3.2.1.8 Can Timeout Duration (second)

Variable	CanTimeoutDuration	
Description	Specifies the maximum time for blocking function until a timeout is detected. Unit is seconds. This Timeout is used to detect the Hardware Errors/ Production Errors. When Hardware registers like Controller Register (CTRL) or Module Control Register(MCR) are configured, the Hardware takes some time to take effect of these new settings CANuusted. Once timeout has been occurred and if hardware could not take effect of the CANu settings, then Error is reported. So this timeout is used to allow hardware to take effect of the Hardware settings.	
Screenshot	Can Timeout Duration (second) (0.001 -> 65.535) 	
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Default	0.1
	Origin	AUTOSAR_ECUC

3.2.1.9 Pretended Networking Activation

Variable	CanPublicComSupport	
Description	Selects support of Pretended Network features in CAN driver. - True: Enabled. - False: Disabled.	
Screenshot	Pretended Networking Activation 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

3.2.1.10 CanSetBaudrateApi

Variable	CanSetBaudrateApi	
Description	ECUC_Can_00482: If the parameter is set to true the Can_SetBaudrate Api shall be supported.	
Screenshot	 CanSetBaudrateApi 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

3.2.1.11 Provide Version Info API

Variable	CanVersionInfoApi	
Description	Switches the Can_GetVersionInfo() API: ON or OFF. When this option is ON driver supports API for getting Version information for the Driver.	
Screenshot	Provide Version Info API 	

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	AUTOSAR_ECUC

3.2.1.12 Can define loop as cycle

Variable	CanAllowLoopAsCycle	
Description	N/A	
Screenshot	Can define loop as cycle   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.1.13 Can Timeout Duration (in Number of loops)

Variable	CanNumberOfTimeOutLoop	
Description	N/A	
Screenshot	Can Timeout Duration (in Number of loops) (1 -> 65535)  1 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip

3.2.1.14 Can Extended ID Support

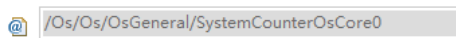
Variable	CanExtendedIDSupport	
Description	Enables support of Extended/Mixed mode (permit to use IDs of 29 bits length). Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Extended ID Support   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip

3.2.1.15 Can MB Count Extension Support

Variable	CanMBCountExtensionSupport	
Description	Enables support of more than 255 Can Hardware Objects. Some platforms have a bigger number of Can controllers and the sum of total MBS for all controllers is bigger than uint8 size (as HTH/HRH is specified in	

	Autosar). This option should not be enabled for platforms that have a number of MBs smaller than 256 (summing all Can controllers from the platform). Note: Implementation Specific parameter.	
Screenshot	Can MB Count Extension Support 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip

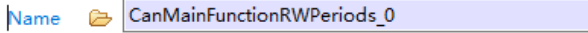
3.2.1.16 Os Counter

Variable	CanOsCounterRef	
Description	This parameter contains a reference to the counter, which is used by the CAN driver. Reference to [OsCounter]	
Screenshot	 	
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.1.17 CanSupportTTCANRef

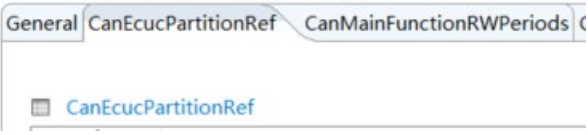
Variable	CanSupportTTCANRef	
Description	Reference to CAN Hardware Object which shall be polled with the configured CanMainFunctionWritePeriod. This reference shall only be configurable if more than one period is configured via CanMainFunctionWritePeriod.	
Screenshot	N/A	
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.1.18 CanMainFunctionRWPeriods

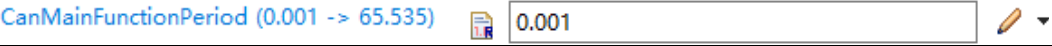
Container	CanMainFunctionRWPeriods	
Description	This container contains the parameter for configuring the period for cyclic call to Can_MainFunction_Read or Can_MainFunction_Write depending on the referring item.	
Screenshot	CanMainFunctionRWPeriods 	
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.1.18.1 CanEcucPartitionRef

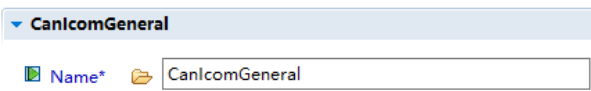
Variable	CanEcucPartitionRef	
Description	Maps the CAN driver to zero or multiple ECUC partitions to make the modules API available in this partition. The CAN driver will operate as an independent instance in each of the partitions.	

Screenshot		
Properties	Property	Value
	Type	REF
	SymbolicName	false
	Default	N/A
	Origin	AUTOSAR_ECUC

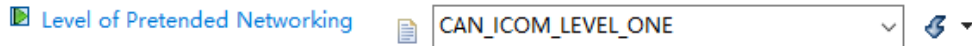
3.2.1.18.2 CanMainFunctionPeriod

Variable	CanMainFunctionPeriod	
Description	This parameter describes the period for cyclic call to Can_MainFunction_Read or Can_MainFunction_Write depending on the referring item. Unit is seconds. Different poll-cycles will be configurable if more than one CanMainFunctionPeriod is configured. In this case, multiple Can_MainFunction_Read() or Can_MainFunction_Write() will be provided by the CAN Driver module.	
Screenshot		
Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Default	0.001
	Origin	AUTOSAR_ECUC

3.2.1.19 CanIcomGeneral

Container	CanIcomGeneral	
Description	This container contains the general configuration parameters of the ICOM Configuration	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

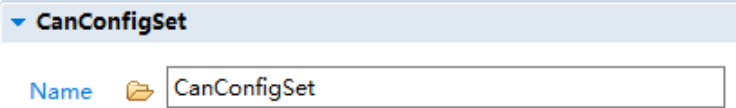
3.2.1.19.1 CanIcomLevel

Variable	CanIcomLevel	
Description	ECUC_Can_00445: Defines the level of Pretended Networking. This parameter is reserved for future implementations (Pretended Networking level 2).	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	CAN_ICOM_LEVEL_ONE
	Origin	AUTOSAR_ECUC

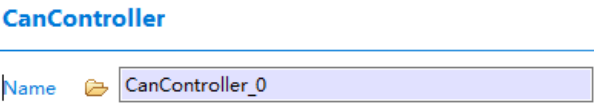
3.2.1.19.2 CanIcomVariant

Variable	CanIcomVariant	
Description	Defines the variant, which is supported by this CanController.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	CAN_ICOM_VARIANT_NONE
	Origin	AUTOSAR_ECUC

3.2.2 CanConfigSet

Container	CanConfigSet	
Description	This is the multiple configuration set container for CAN Driver.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1 CanController

Container	CanController	
Description	This container contains the configuration parameters of the CAN controller(s).	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.1 CanHwChannel

Variable	CanHwChannel	
Description	Specifies which one of the on-chip FlexCAN interfaces is associated with this controller ID. Note: Implementation Specific parameter. Not AutoSar Required.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Origin	Flagchip

3.2.2.1.2 CanBusoffProcessing

Variable	CanBusoffProcessing	
Description	Enables/Disables API Can_MainFunction_BusOff() for handling busoff events in POLLING mode.	
Screenshot		

Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	POLLING
	Origin	AUTOSAR_ECUC

3.2.2.1.3 Can Controller Activation

Variable	CanControllerActivation	
Description	Defines if a CAN controller is used in the configuration. Deactivation of a particular CAN controller is equivalent to a CAN controller not used in the configuration.	
Screenshot	Can Controller Activation 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	AUTOSAR_ECUC

3.2.2.1.4 Can Controller Base Address

Variable	CanControllerBaseAddress	
Description	Specifies the CAN controller base address.	
Screenshot	Can Controller Base Address (0 -> 4294967295) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

3.2.2.1.5 Can Controller ID

Variable	CanControllerId	
Description	This parameter provides the controller ID which is unique in a given CAN Driver. The value for this parameter starts with 0 and continue without any gaps.	
Screenshot	Can Controller ID 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	true
	Origin	AUTOSAR_ECUC

3.2.2.1.6 Can Rx Processing Type

Variable	CanRxProcessing	
Description	Enables/Disables API Can_MainFunction_Read() for handling PDU reception events in POLLING mode.	
Screenshot	Can Rx Processing Type 	

Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	POLLING
	Origin	AUTOSAR_ECUC

3.2.2.1.7 Can Tx Processing Type

Variable	CanTxProcessing	
Description	Enables/Disables API Can_MainFunction_Write() for handling PDU transmission events in POLLING mode	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	POLLING
	Origin	AUTOSAR_ECUC

3.2.2.1.8 CanWakeupFunctionalityAPI

Variable	CanWakeupFunctionalityAPI	
Description	Adds / removes the service Can_CheckWakeup() from the code. True: Can_CheckWakeup can be used. False: Can_CheckWakeup cannot be used.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

3.2.2.1.9 Can Wakeup Processing Type

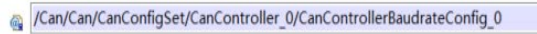
Variable	CanWakeupProcessing	
Description	Enables/Disables API Can_MainFunction_Wakeup() for handling wakeup events in POLLING mode. Note: This option is enabled only if global parameter is 'true'.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	POLLING
	Origin	AUTOSAR_ECUC

3.2.2.1.10 Can Wakeup Support

Variable	CanWakeupSupport
----------	------------------

Description	CAN driver supports for wakeup over CAN Bus. Every WakeUp process will be ignored if this checkbox is not set to ON. This parameter enables Internal Wakeup (using controller registers) and External Wakeup (using WKUP module). This is enabled only if internal Wakeup is supported by the platform.	
Screenshot	Can Wakeup Support 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

3.2.2.1.11 Can Controller Default Baudrate

Variable	CanControllerDefaultBaudrate	
Description	Reference to baudrate configuration container configured for the Can Controller.	
Screenshot	Can Controller Default Baudrate 	
Properties	Property	Value
	Type	Reference
	SynbolicName	false
	Default	N/A
	Origin	AUTOSAR_ECUC

3.2.2.1.12 CanControllerEcucPartitionRef

Variable	CanControllerEcucPartitionRef	
Description	Maps the CAN controller to zero or one ECUC partitions. The ECUC partition referenced is a subset of the ECUC partitions where the CAN driver is mapped to.	
Screenshot	 CanControllerEcucPartitionRef 	
Properties	Property	Value
	Type	Reference
	SynbolicName	false
	Default	N/A
	Origin	AUTOSAR_ECUC

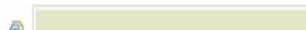
3.2.2.1.13 Can CPU Module Reference Clock

Variable	CanCpuModuleClockRef	
Description	Reference to the CPU Module clock configuration	
Screenshot	Can CPU Module Reference Clock 	
Properties	Property	Value
	Type	Reference
	SynbolicName	false
	Default	N/A
	Origin	AUTOSAR_ECUC

3.2.2.1.14 Can CPU Function Reference Clock

Variable	CanCpuFunctionClockRef	
Description	Reference to the CPU function clock configuration, which is set in the MCU driver configuration	
Screenshot	 	
Properties	Property	Value
	Type	Reference
	SynmbolicName	false
	Default	N/A
	Origin	AUTOSAR_ECUC

3.2.2.1.15 CanWakeupSourceRef

Variable	CanCpuModuleClockRef	
Description	This parameter contains a reference to the Wakeup Source for this controller as defined in the ECU State Manager. Type: reference to EcuM_WakeupSourceType EcuM plugin need to be added and then give the reference to it.	
Screenshot	 	
Properties	Property	Value
	Type	Reference
	SynmbolicName	false
	Default	N/A
	Origin	AUTOSAR_ECUC

3.2.2.1.16 Can Controller Icom Supported

Variable	CanControllerIcomSupported	
Description	Define if the current controller support Pretended Networking (Icom).	
Screenshot	 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.17 Can BCC Enable

Variable	CanBccSupport	
Description	Defines if Backwards Compatibility Configuration (BCC) feature of CAN controller is used in the configuration. If BCC feature of CAN controller is enabled, Individual Rx masking and queue feature are disabled. If BCC feature of CAN controller is disabled, Individual Rx masking and queue feature are enabled. Note: Implementation Specific parameter.	
Screenshot	 	

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.18 Can Listen Only Mode (CTRL[LOM])

Variable	CanListenOnlyMode	
Description	Enables the Listen Only Mode (CTRL[LOM] bit). In this mode, transmission is disabled, all error counters are frozen and the module operates in a CAN Error Passive mode. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Listen Only Mode (CTRL[LOM])   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.19 Can Loop Back Mode (CTRL[LPB])

Variable	CanLoopBackMode	
Description	CTRL[LPB]: Enables CAN to operate in Loop Back Mode. The bit stream output of the transmitter is fed back internally to the receiver input. The Rx CAN input pin is ignored and the Tx CAN output goes to the recessive state (logic "1"). Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Loop Back Mode (CTRL[LPB])   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.20 Can Enable Software BusOff Recovery

Variable	CanSoftwareBusOffRecovery	
Description	Enables Software Bus Off recovery when automatic recovering from BusOff state is disabled for CAN controller. Note: Implementation specific Parameter. This parameter is enabled when CanAutoBusOffRecovery is set to false.	
Screenshot	Can Enable Software BusOff Recovery   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.21 Can Auto BusOff Recovery (CTRL[BOFF_REC])

Variable	CanAutoBusOffRecovery	
Description	Enable/Disable automatic BusOff recovery (CTRL[BOFF_REC] bit). 0(Checked) = Automatic recovering from Bus Off state occurs according to the CAN Specification 2.0B. 1(Unchecked) = Automatic recovering from Bus Off is disabled and the module remains in Bus Off state until the bit is negated(zero) by the user. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Auto BusOff Recovery (CTRL[BOFF_REC])    	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.22 Can Triple Sampling Mode Enable (CTRL[SMP])

Variable	CanTrippleSamplingEnable	
Description	CTRL[SMP]: This bit defines the sampling mode of CAN bits at the Rx input. 1 - Enables acquisition of 3 samples and majority voting for the value of received bit. 0 - Just one sample is used to determine the bit value. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Triple Sampling Mode Enable (CTRL[SMP])    	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.23 Can WakeUp Source Filter (MCR[WAK_SRC])

Variable	CanWakeUpSourceFilter	
Description	Defines whether the integrated low-pass filter is applied to protect the Rx CAN input from spurious wake up. 1 = FlexCAN uses the filtered Rx input to detect recessive to dominant edges on the CAN bus. 0 = FlexCAN uses the unfiltered Rx input to detect recessive to dominant edges on the CAN bus. This option will provide the setting for init the controller with MCR[WAK_SRC] bit set. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	N/A	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.24 Can Lowest Buffer Transmit First (CTRL[LBUF])

Variable	CanLowestBuffTransmitFirst
-----------------	----------------------------

Description	CTRL[LBUF]. This bit defines the ordering mechanism for Message Buffer transmission. When asserted, the MCR[LPRIO_EN] bit doesn't affect the priority arbitration. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Lowest Buffer Transmit First (CTRL[LBUF])   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip

3.2.2.1.25 Can Local Priority Enable (MCR[LPRIO_EN])

Variable	CanLocalPriorityEn	
Description	MCR[LPRIO_EN]. This bit controls whether the local priority feature is enabled or not. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Local Priority Enable (MCR[LPRIO_EN])   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.26 Can Clock from Bus (CTRL[CLK_SRC])

Variable	CanClockFromBus	
Description	Switches the source clock for the module to the system bus (rather than crystal). 1 = The CAN engine clock source is the bus clock. (from MCU) 0 = The CAN engine clock source is the oscillator clock. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can Clock from Bus (CTRL[CLK_SRC])   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip

3.2.2.1.27 Can FD Enable

Variable	CanControllerFdEnable	
Description	Enable CAN FD.	
Screenshot	Can FD Enable   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.28 Can FD Mode

Variable	CanControllerFdType	
Description	ISO_CAN_FD BOSCH_CAN_FD	
Screenshot	Can FD Mode 	
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	ISO_CAN_FD
	Origin	Flagchip
	Range	ISO_CAN_FD, BOSCH_CAN_FD

3.2.2.1.29 Can Legacy RxFifo Enable

Variable	CanControllerLegacyRxFifoEnable	
Description	Defines if RX FIFO feature of CAN controller is used in the configuration. If FIFO feature of CAN controller is enabled and can FD is disabled, Legacy MB First 8 Message Buffers will be used by FIFO engine. If FIFO feature of CAN controller is enabled and can FD is enabled, Enhanced FIFO will be used by FIFO engine.	
Screenshot	Can Legacy RxFifo Enable 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.30 Can FD Enhanced RxFifo Enable

Variable	CanControllerEnhancedRxFifoEnable	
Description	Defines if Enhanced FIFO feature of CAN controller is used in the configuration. Only used in CAN FD Mode.	
Screenshot	N/A	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.31 Can Legacy Message buffer data size(FDCTRL[MBDSR0])

Variable	CanLegacyMessageBufferDataBlockSize	
Description	Selects maximum payload of MB's.	
Screenshot	Can Legacy Message buffer data size(FDCTRL[MBDSR0]) 	
Properties	Property	Value
	Type	ENUMERATION
	Default	CAN_8_BYTES_PAYLOAD

	Origin	Flagchip
	Range	CAN_8_BYTES_PAYLOAD, CAN_16_BYTES_PAYLOAD, CAN_32_BYTES_PAYLOAD, CAN_64_BYTES_PAYLOAD

3.2.2.1.32 Can Legacy Maximum Message Buffers

Variable	CanLegacyMaxMessageBuffersCnt	
Description	This parameter describes the maximum Message Buffer Number of CAN Legacy MB.	
Screenshot	 21 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	Flagchip

3.2.2.1.33 Can Error Notification Enable

Variable	CanErrorControllerNotifEn	
Description	Enables/Disables the Error Controller Notification. If Disabled, no error INTERRUPT or notification shall take place. Note: Implementation Specific parameter. Not AutoSar Required.	
Screenshot	  ☒ 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip

3.2.2.1.34 CanErrorControllerNotification

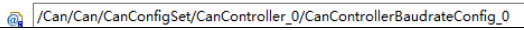
Variable	CanErrorControllerNotification	
Description	Set here the name of the Error INTERRUPT controller notification function. Note: Implementation Specific parameter.	
Screenshot	 NULL_PTR 	
Properties	Property	Value
	Type	FUNCTION-NAME
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip

3.2.2.1.35 CanErrorFDControllerNotification

Variable	CanErrorFDControllerNotification	
Description	Set here the name of the Error INTERRUPT controller notification function for errors detected in the Data Phase of CAN FD frames with the BRS bit set only. Note: Implementation Specific parameter.	
Screenshot	 NULL_PTR 	
Properties	Property	Value

	Type	FUNCTION-NAME
	SymbolicName	false
	Default	NULL_PTR
	Origin	Flagchip

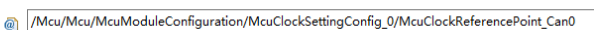
3.2.2.1.36 Can Controller Default Baudrate

Variable	CanControllerDefaultBaudrate	
Description	Reference to baudrate configuration container configured for the Can Controller.	
Screenshot	 	
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

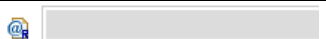
3.2.2.1.37 Can CPU Module Reference Clock

Variable	CanCpuModuleClockRef	
Description	Reference to the CPU Module clock configuration.	
Screenshot	 	
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.38 Can CPU Function Reference Clock

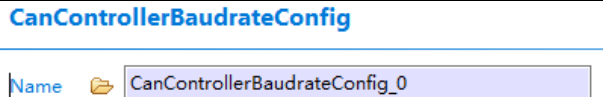
Variable	CanCpuFunctionClockRef	
Description	Reference to the CPU function clock configuration, which is set in the MCU driver configuration.	
Screenshot	 	
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.39 CanWakeupSourceRef

Variable	CanWakeupSourceRef	
Description	This parameter contains a reference to the Wakeup Source for this controller as defined in the ECU State Manager. Type: reference to EcuM_WakeupSourceType EcuM plugin need to be added and then give the reference to it.	
Screenshot	 	
Properties	Property	Value
	Type	SYMBOLIC-NAME-REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.40 CanControllerBaudrateConfig

Container	CanControllerBaudrateConfig
------------------	-----------------------------

Description	This container contains bit timing related configuration parameters of the CAN controller(s).	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.40.1 Can Module Clock (Hz) - Reselect after clock change

Variable	CanModuleClock	
Description	Can baudrate source clock, if CLK_SRC set 1, is from Module clock, or is from function clock (Divided from OSC by PCC).	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	Flagchip
	Range	Mcu Clock Reference Point

3.2.2.1.40.2 Can Controller BaudRate (Kbps)

Variable	CanControllerBaudRate	
Description	Specifies the buadrate of the controller in kbps. CAN maximum speed is 1Mbps.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	500
	Origin	AUTOSAR_ECUC
	Range	0-2000

3.2.2.1.40.3 Can Controller BaudRate Config ID

Variable	CanControllerBaudRateConfigID	
Description	Uniquely identifies a specific baud rate configuration. This ID is used by SetBaudrate API.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	AUTOSAR_ECUC
	Range	0-65535

3.2.2.1.40.4 Can Controller Prescaler

Variable	CanControllerPrescaler
-----------------	------------------------

Description	Specifies the prescaler for the controller. The calculation of the resulting CanControllerTimeQuanta value depending on module clocking and prescaler shall be done offline. $\text{Prescaler} = \text{FreqCanClk} / \text{FreqTq}$; $\text{FreqTq} = 1 / \text{CanControllerTimeQuanta}$.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	3
	Origin	Flagchip

3.2.2.1.40.5 Can Total Needed Tq number

Variable	CanNeedTqs	
Description	Total Can Baud Tq num, and may set it in range 8-25 generally.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	Flagchip
	Range	8-25

3.2.2.1.40.6 Can Synchronization Segment

Variable	CanControllerSyncSeg	
Description	The Synchronization Segment or SYNC_SEG time interval is used to synchronize all the nodes across the network. The SYNC_SEG time interval has a fixed period of one Time Quantum (TQ).	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip

3.2.2.1.40.7 Can Propagation Segment

Variable	CanControllerPropSeg	
Description	It is used to compensate the physical delay within the CAN network (CTRL[PROPSEG] - 0..7).	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	7
	Origin	AUTOSAR_ECUC
	Range	1-64

3.2.2.1.40.8 Can Phase Segment 1

Variable	CanControllerSeg1	
Description	Specifies the Phase Segment 1 in time quantas (CTRL[PSEG1] = 0..7). CanControllerSeg1 = (PSEG1 + 1) * Tq. The CanControllerSeg1 valid values are 1-8 Tq.	
Screenshot	Can Phase Segment 1 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	AUTOSAR_ECUC
	Range	1-32

3.2.2.1.40.9 Can Phase Segment 2

Variable	CanControllerSeg2	
Description	Specifies the Phase Segment 1 in time quantas (CTRL[PSEG2] = 1..7). CanControllerSeg2 = (PSEG2 + 1) * Tq. The CanControllerSeg2 valid values are 2-8 Tq.	
Screenshot	Can Phase Segment 2 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	AUTOSAR_ECUC
	Range	1-32

3.2.2.1.40.10 Can Resynch Jump Width

Variable	CanControllerSyncJumpWidth	
Description	Specifies the synchronization jump width for the controller in time quantas.	
Screenshot	Can Resynch Jump Width 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	AUTOSAR_ECUC
	Range	1-4(FD disabled), 1-32(FD enabled)

3.2.2.1.40.11 Can FD Sample Point

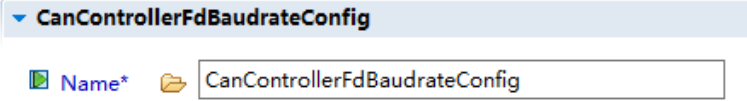
Variable	CanSamplePoint	
Description	CAN baudrate source clock, if CLK_SRC set 1, is from Module clock, or is from function clock (Divided from OSC by PCC)	
Screenshot	Can FD Sample Point 	

Properties	Property	Value
	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
	Range	0-1

3.2.2.1.40.12 CanTxArbitrationStart Delay

Variable	CanTxArbitrationStartDelay	
Description	This 5-bit field indicates how many CAN bits the Tx arbitration process start point can be delayed from the first bit of CRC field on CAN bus. See Reference Manual to have a calculation method for the optimal T ASD value.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	20
	Origin	Flagchip

3.2.2.1.40.13 CanControllerFdBaudrateConfig

Container	CanControllerFdBaudrateConfig	
Description	This optional container contains bit timing related configuration parameters of the CAN controller(s) for payload and CRC of a CAN FD frame. If this container exists, the controller supports CAN FD frames.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

Can FD Controller BaudRate (Kbps)

Variable	CanControllerFdBaudRate	
Description	Specifies the data segment baud rate of the controller in kbps.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1000
	Origin	AUTOSAR_ECUC

Can FD Controller Prescaler

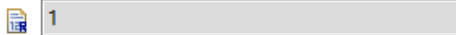
Variable	CanControllerFdPrescaler	
-----------------	--------------------------	--

Description	Specifies the FD prescaler for the controller. The calculation of the resulting CanControllerFdTimeQuanta value depending on module clocking and prescaler shall be done offline. $\text{Prescaler} = \text{FreqCanClk} / \text{FreqTq}$; $\text{FreqTq} = 1 / \text{CanControllerTimeQuanta}$.	
Screenshot	Can FD Controller Prescaler (1 -> 1024) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	3
	Origin	Flagchip

Can FD Total Needed Tq number

Variable	CanFdNeedTqs	
Description	CAN baudrate source clock, if CLK_SRC set 1, is from Module clock, or is from function clock (Divided from OSC by PCC).	
Screenshot	Can FD Total Needed Tq number 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Origin	Flagchip
	Range	5-48

Can FD Synchronization Segment

Variable	CanControllerFdSyncSeg	
Description	The Synchronization Segment or SYNC_SEG time interval is used to synchronize all the nodes across the network. The SYNC_SEG time interval has a fixed period of one Time Quantum (TQ).	
Screenshot	Can FD Synchronization Segment (1 -> 1) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	Flagchip

Can FD Propagation Segment

Variable	CanControllerPropSeg	
Description	Specifies propagation delay in time quantas.	
Screenshot	Can FD Propagation Segment 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	3
	Origin	AUTOSAR_ECUC

	Range	1-31
--	-------	------

Can FD Phase Segment 1

Variable	CanControllerSeg1	
Description	Specifies phase segment 1 in time quantas.	
Screenshot	Can FD Phase Segment 1  2 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	AUTOSAR_ECUC
	Range	1-8

Can FD Phase Segment 2

Variable	CanControllerSeg2	
Description	Specifies phase segment 2 in time quantas.	
Screenshot	Can FD Phase Segment 2  2 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	2
	Origin	AUTOSAR_ECUC
	Range	1-8

Can FD Resynch Jump Width

Variable	CanControllerSyncJumpWidth	
Description	Specifies the synchronization jump width for the controller in time quantas.	
Screenshot	Can FD Resynch Jump Width  4 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	4
	Origin	AUTOSAR_ECUC
	Range	1-8

Can FD Sample Point

Variable	CanFdSamplePoint	
Description	Can baudrate source clock, if CLK_SRC set 1, is from Module clock, or is from function clock (Divided from OSC by PCC)	
Screenshot	Can FD Sample Point  0.81818181818182 	
Properties	Property	Value

	Type	FLOAT
	SymbolicName	false
	Origin	Flagchip
	Range	0-1

Controller Trcv Delay Compensation Offset

Variable	CanControllerTrcvDelayCompensationOffset	
Description	Specifies the Transceiver Delay Compensation Offset in ns. If not specified Transceiver Delay Compensation is disabled.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC
	Range	0-400

CanControllerTxBitRateSwitch

Variable	CanControllerTxBitRateSwitch	
Description	Specifies if the bit rate switching shall be used for transmissions.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	AUTOSAR_ECUC

CanControllerFdIsoCANFD

Variable	CanControllerFdIsoCANFD	
Description	Add support for selecting ISO or non ISO for CAN FD feature.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	true
	Origin	Flagchip

CanControllerFdPrExcEn

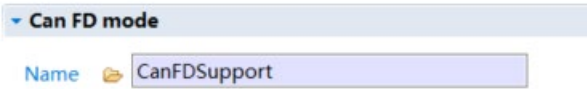
Variable	CanControllerFdPrExcEn	
Description	This bit enables the Protocol Exception feature.	
Screenshot		

Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

CanControllerFdEdgeFilterDis

Variable	CanControllerFdEdgeFilterDis	
Description	This bit disables the Edge Filter.	
Screenshot	CanControllerFdEdgeFilterDis 	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

3.2.2.1.41 CAN FD mode

Container	CanFDSupport	
Description	This container contains the configuration (parameters) of the CANFD.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.41.1 Can FD Support

Variable	CanControllerLegacyIDAcceptanceMode	
Description	Enable CAN FD.	
Screenshot	Can FD Support 	
Properties	Property	Value
	Type	Boolean(Range)
	SynmbolicName	false
	Default	false
	Origin	Flagchip
	Range	[true, false]

3.2.2.1.41.2 Can FD Mode

Variable	CanControllerFdType	
Description	ISO_CAN_FD BOSCH_CAN_FD	
Screenshot	Can FD Mode 	
Properties	Property	Value

	Type	String(Range)
	SynmbolicName	false
	Default	ISO_CAN_FD
	Origin	Flagchip
	Range	[ISO_CAN_FD, BOSCH_CAN_FD]

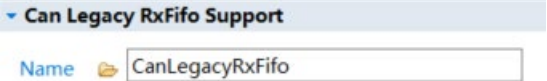
3.2.2.1.41.3 Can FD Message buffer data size(FDCTRL[MBDSR0])

Variable	CanLegacyMessageBufferDataBlockSize	
Description	Selects maximum payload of MB's.	
Screenshot		
Properties	Property	Value
	Type	String(Range)
	SynmbolicName	false
	Default	CAN_8_BYTES_PAYLOAD
	Origin	Flagchip
	Range	[CAN_8_BYTES_PAYLOAD, CAN_16_BYTES_PAYLOAD, CAN_32_BYTES_PAYLOAD, CAN_64_BYTES_PAYLOAD]

3.2.2.1.41.4 Can FD Maximum Message Buffers

Variable	CanLegacyMaxMessageBuffersCnt	
Description	This parameter describes the maximum Message Buffer Number of CAN Legacy MB.	
Screenshot		
Properties	Property	Value
	Type	Integer
	SynmbolicName	false
	Default	N/A
	Origin	Flagchip
	Range	N/A

3.2.2.1.42 CanLegacyRxFifo

Container	Can Legacy RxFifo Support	
Description	This container contains the configuration (parameters) of the CAN Legacy RX FIFO. Note: This container is ENABLED only when CanControllerLegacyRxFifoEnable is TRUE and CanControllerLegacyRxFifoEnable is FALSE.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.42.1 Can Legacy RxFifo Enable

Variable	CanControllerLegacyRxFifoEnable
-----------------	---------------------------------

Description	Defines if RX FIFO feature of CAN controller is used in the configuration. If FIFO feature of CAN controller is enabled and can fd is disabled, Legacy MB First 8 Message Buffers will be used by FIFO engine. If FIFO feature of CAN controller is enabled and can fd is enabled, Enhanced FIFO will be used by FIFO engine.	
Screenshot		
Properties	Property	Value
	Type	Boolean(Range)
	SynmbolicName	false
	Default	false
	Origin	Flagchip
	Range	[true, false]

3.2.2.1.42.2 Hardware Object Uses Polling

Variable	CanHardwareObjectUsesPolling	
Description	Enables polling of this fifo hardware object. This node shall exist if CanRxProcessing is set to MIXED.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.42.3 Can MainFunction RW Period Reference

Variable	CanMainFunctionRWPeriodRef	
Description	Reference to CAN Controller to which the HOH is associated to.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.42.4 Can Legacy RxFifo ID Acceptance Mode

Variable	CanControllerLegacyIDAcceptanceMode	
Description	This 2-bit field identifies the format of the elements of the Legacy Rx FIFO filter table. NotImplementation Specific parameter. Format Explanation A : One full ID (standard or extended) per filter element. B : Two full standard IDs or two partial 14-bit extended IDs per filter element. C : Four partial 8-bit IDs (standard or extended) per filter element. D : All frames rejected. Note: The TableID and Filter Mask values don't care for this option.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	A
	Origin	Flagchip

	Range	
--	-------	--

3.2.2.1.42.5 Can Legacy RxFifo Filters Number

Variable	CanLegacyRxFifoFiltersNumber	
Description	This field defines the number of Rx FIFO filters. NoteImplementation Specific parameter.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	FILTERS_NUMBER_8
	Origin	Flagchip
	Range	FILTERS_NUMBER_8 FILTERS_NUMBER_16 FILTERS_NUMBER_24 FILTERS_NUMBER_32 FILTERS_NUMBER_40 FILTERS_NUMBER_48 FILTERS_NUMBER_56 FILTERS_NUMBER_64 FILTERS_NUMBER_72 FILTERS_NUMBER_80 FILTERS_NUMBER_88 FILTERS_NUMBER_96 FILTERS_NUMBER_104 FILTERS_NUMBER_112 FILTERS_NUMBER_120 FILTERS_NUMBER_128

3.2.2.1.42.6 Can Legacy RxFifo Global Filter Mask

Variable	CanLegacyRxFifoGlobalMaskValue	
Description	Specifies Rx Fifo Global Filter Mask value to be used as acceptance criteria for the Rx Fifo filter IDs which are not covered by individual masks. NoteImplementation Specific parameter. Value for this parameter should be entered based on the CanControllerLegacyIDAcceptanceMode selected as explained below. A : Filtermask value for One full standard or Extended ID based on the CanTableIDType selected. B : Filtermask value for CanIDValue0(bit field: 0-11 for standard ID type and 0-14 for extended ID type) and CanIDValue1 (bit field: 12-22 for standard ID type and 15-28 for extended ID type). C : Filtermask value for CanIDValue0(bit field: 0-8), CanIDValue1(bit field: 9-16), CanIDValue2(bit field: 17-24) and CanIDValue3(bit field: 25-32). It shall be distinguished between: - Standard identifier mask - Extended identifier mask 0 - the corresponding bit in the filter is "don't care". 1 - the corresponding bit in the filter is used during the matching process.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	2047
	Origin	Flagchip

3.2.2.1.42.7 CanRxFifoWarningNotification

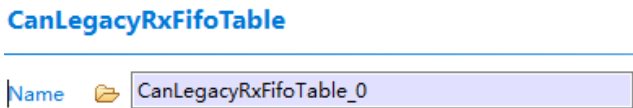
Variable	CanRxFifoWarningNotification	
Description	Set here the name of the handler for RxFifo Warning notification function. Note: Implementation Specific parameter	
Screenshot		

Properties	Property	Value
	Type	FUNCTION-NAME
	SynmbolicName	false
	Default	NULL_PTR
	Origin	Flagchip

3.2.2.1.42.8 CanRxFifoOverflowNotification

Variable	CanRxFifoOverflowNotification	
Description	Set here the name of the handler for RxFifo Overflow notification function. NoteImplementation Specific parameter	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	SynmbolicName	false
	Default	NULL_PTR
	Origin	Flagchip

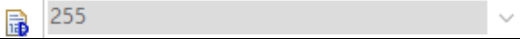
3.2.2.1.42.9 CanLegacyRxFifoTable

Container	CanLegacyRxFifoTable	
Description	This container contains a multiple entry ID table information required to configure RX FIFO. The number of entries shall be equal to the number of RxFifo filters configured using CanLegacyRxFifoFiltersNumber parameter.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

Can RxFifo Identifier 0

Variable	CanIDValue0	
Description	Specifies an ID to be used as acceptance criteria for the ID Table 0. NoteImplementation Specific parameter. Value for this parameter should be entered based on the CanControllerLegacyIDAceptanceMode selected as explained below. A : One full ID (standard or extended) per filter element. B : One full standard ID if the CanTableIDType is Standard or one 14 most significant bit value of Extended ID if the CanTableIDType is Extended. C : One 8 most significant bit value of Standard or Extended ID.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	255
	Origin	Flagchip

Can RxFifo Identifier 1

Variable	CanIDValue1	
Description	Specifies an ID to be used as acceptance criteria for the ID Table. NoteImplementation Specific parameter. Value for this parameter should be entered based on the CanControllerLegacyIDAcceptanceMode selected as explained below. B : One full standard ID if the CanTableIDType is Standard or one 14 most significant bit value of Extended ID if the CanTableIDType is Extended. C : One 8 most significant bit value of Standard or Extended ID.	
Screenshot	Can RxFifo Identifier 1 	
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	255
	Origin	Flagchip

Can RxFifo Identifier 2

Variable	CanIDValue2	
Description	Specifies an ID to be used as acceptance criteria for the ID Table. NoteImplementation Specific parameter. Value for this parameter should be entered based on the CanControllerLegacyIDAcceptanceMode selected as explained below. C : One 8 most significant bit value of Standard or Extended ID.	
Screenshot	Can RxFifo Identifier 2 	
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	255
	Origin	Flagchip

Can RxFifo Identifier 3

Variable	CanIDValue3	
Description	Specifies an ID to be used as acceptance criteria for the ID Table. NoteImplementation Specific parameter. Value for this parameter should be entered based on the CanControllerLegacyIDAcceptanceMode selected as explained below. C : One 8 most significant bit value of Standard or Extended ID.	
Screenshot	Can RxFifo Identifier 3 	
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	255
	Origin	Flagchip

Can RxFifo Frame Accepted Mode

Variable	CanTableIDType
-----------------	----------------

Description	Specifies whether extended or standard frames are accepted into the FIFO. NoteImplementation Specific parameter.	
Screenshot	Can RxFifo Frame Accepted Mode 	
Properties	Property	Value
	Type	ENUMERATION
	SynmbolicName	false
	Default	Standard
	Origin	Flagchip
	Range	Standard, Extended

Can RxFifo Filter Mask

Variable	CanMBFilterMaskValue	
Description	Specifies Filetr Mask value to be used as acceptance criteria for the ID Table. NoteImplementation Specific parameter. Value for this parameter should be entered based on the CanControllerLegacyIDAcceptanceMode selected as explained below. A : Filtermask value for One full standard or Extended ID based on the CanTableIDType selected. B : Filtermask value for CanIDValue0(bit field: 0-11 for standard ID type and 0-14 for extended ID type) and CanIDValue1 (bit field: 12-22 for standard ID type and 15-28 for extended ID type). C : Filtermask value for CanIDValue0(bit field: 0-8), CanIDValue1(bit field: 9-16), CanIDValue2(bit field: 17-24) and CanIDValue3(bit field: 25-32).	
Screenshot	Can RxFifo Filter Mask 	
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	2047
	Origin	Flagchip
	Range	0- 4294967295

3.2.2.1.43 CanEnhancedRxFifo

Container	CanEnhancedRxFifo	
Description	This container contains the configuration (parameters) of the CAN Enhanced RX FIFO. Note: This container is ENABLED only when CAN FD is enabled.	
Screenshot	N/A	
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.1.43.1 Can FD Enhanced RxFifo Enable

Variable	CanControllerEnhancedRxFifoEnable	
Description	Defines if Enhanced FIFO feature of CAN controller is used in the configuration. Only used in CAN FD Mode.	
Screenshot	Can Legacy RxFifo Enable 	
Properties	Property	Value
	Type	IDENTIFIABLE

	Type	Boolean(Range)
	SynmbolicName	false
	Default	false
	Origin	Flagchip
	Range	[true, false]

3.2.2.1.43.2 Hardware Object Uses Polling

Variable	CanHardwareObjectUsesPolling	
Description	Enables polling of this fifo hardware object. This node shall exist if CanRxProcessing is set to MIXED.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.43.3 Can MainFunction RW Period Reference

Variable	CanMainFunctionRWPeriodRef	
Description	Reference to CAN Controller to which the HOH is associated to.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.1.43.4 Can standard ID Filter Number

Variable	CanStandardIDFilterNumber	
Description	CAN Enhanced Rx FIFO standard ID filter Number NOTICE: standard ID filter Number + 2 * Extended ID filter Number <= 32	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	2
	Origin	Flagchip
	Range	N/A

3.2.2.1.43.5 Can Extended ID Filter Number

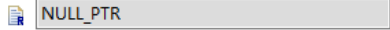
Variable	CanExtendedIdFilterNumber	
Description	CAN Enhanced Rx FIFO Extended ID filter Number NOTICE: standard ID filter Number + 2 * Extended ID filter Number <= 32	
Screenshot		
Properties	Property	Value
	Type	INTEGER

	SynbolicName	false
	Default	0
	Origin	Flagchip
	Range	N/A

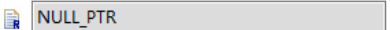
3.2.2.1.43.6 Can FIFO Watermark Number

Variable	CanFifoWatermark	
Description	This field defines the minimum number of CAN messages stored in the Enhanced RX FIFO. When that number is reached, then Interrupt or DMA request would be generated if the interrupt or DMA is enabled.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynbolicName	false
	Default	1
	Origin	Flagchip
	Range	N/A

3.2.2.1.43.7 CanRxFifoWarningNotification

Variable	CanRxFifoWarningNotification	
Description	Set here the name of the handler for RxFifo Warning notification function. NoteImplementation Specific parameter	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	SynbolicName	false
	Default	NULL_PTR
	Origin	Flagchip

3.2.2.1.43.8 CanRxFifoOverflowNotification

Variable	CanRxFifoOverflowNotification	
Description	Set here the name of the handler for RxFifo Overflow notification function. NoteImplementation Specific parameter	
Screenshot		
Properties	Property	Value
	Type	FUNCTION-NAME
	SynbolicName	false
	Default	NULL_PTR
	Origin	Flagchip

3.2.2.1.43.9 CanEnhancedRxFifoTable

Container	CanEnhancedRxFifoTable	
Description	CAN Enhanced Rx FIFO ID Filter Table	
Screenshot	N/A	
Properties	Property	Value
	Type	IDENTIFIABLE

Filter ID

Variable	FilterId	
Description	Holds the handle ID of filter.	
Screenshot		
Properties	Property	Value
	Type	Integer(Range)
	Default	0
	Range	0 -> 32

Can Enhanced FIFO Filter ID Type

Variable	CANEnhancedFifoFilterIdType	
Description	N/A	
Screenshot	N/A	
Properties	Property	Value
	Type	ENUMERATION
	Default	Standard
	Range	Standard, Extended

Can Enhanced FIFO Filter Type

Variable	CANEnhancedFifoFilterType	
Description	N/A	
Screenshot	N/A	
Properties	Property	Value
	Type	ENUMERATION
	Default	Mask
	Range	Mask, Range, TwoMatch

Can Enhanced FIFO RTR Filter 1

Variable	CanRtrFilter1	
Description	Can RTR filter 1.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	0
	Origin	Flagchip

Can Enhanced FIFO RTR Filter 2

Variable	CanRtrFilter2	
Description	Can RTR filter 2.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	0
	Origin	Flagchip

Can Enhanced FIFO Filter RTR Mask 1

Variable	CanRtrFilterMask1	
Description	Can RTR Mask.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	0
	Origin	Flagchip

Can Enhanced FIFO ID Filter 1

Variable	CanIDFilter1	
Description	Can ID filter 1.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	0
	Origin	Flagchip

Can Enhanced FIFO ID Filter 2

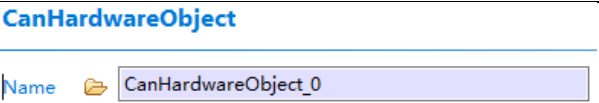
Variable	CanIDFilter2	
Description	Can ID filter2.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynmbolicName	false
	Default	0
	Origin	Flagchip

Can Enhanced FIFO ID MASK 1

Variable	CanIDMask1
-----------------	------------

Description	Can ID mask.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SynbolicName	false
	Default	0
	Origin	Flagchip

3.2.2.2 CanHardwareObject

Container	CanHardwareObject	
Description	This container contains the configuration (parameters) of CAN Hardware Objects. This configuration element is used as information for the CAN Interface only. The relevant CAN driver configuration is done with the filter mask and identifier.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.2.1 FD padding value

Variable	CanFdPaddingValue	
Description	Specifies the value which is used to pad unspecified data in CAN FD frames > 8 bytes for transmission. This is necessary due to the discrete possible values of the DLC if > 8 bytes. If the length of a PDU which was requested to be sent does not match the allowed DLC values, the remaining bytes up to the next possible value shall be padded with this value. MBCS[PRI0]: This value it is the padding value when FD it is used.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

3.2.2.2.2 Can Implementation Type

Variable	CanHandleType	
Description	Specifies the type (Full-CAN or Basic-CAN) of a hardware object. Note: All controllers which the FIFO is enabled shall define at least 1 RECEIVE hardware object. First RECEIVE hardware object defined for a controller which have the FIFO enabled is configured by CONVENTION to receive data from FIFO.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	BASIC

	Origin	AUTOSAR_ECUC
	Range	BASIC, FULL

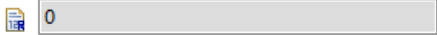
3.2.2.2.3 Can Hardware Object Count

Variable	CanHwObjectCount	
Description	Number of hardware objects used to implement one HOH. In case of a HRH this parameter defines the number of elements in the hardware FIFO or the number of shadow buffers, in case of a HTH it defines the number of hardware objects used for multiplexed transmission or for a hardware FIFO used by a FullCAN HTH	
Screenshot	Can Hardware Object Count (1 -> 65535) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	AUTOSAR_ECUC

3.2.2.2.4 Can ID Message Type

Variable	CanIdType	
Description	Specifies whether the IdValue is of type - standard identifier (ID - 11 bits length) - extended identifier (ID - 29 bits length) - mixed mode (standard or extended) Note: MBs configured as MIXED standard and RECEIVE type will be treated as EXTENDED.	
Screenshot	Can ID Message Type 	
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	STANDARD
	Origin	AUTOSAR_ECUC
	Range	EXTENDED, MIXED, STANDARD

3.2.2.2.5 Can ID Bits Local Priority

Variable	CanMBPrio	
Description	MBCS[PRI0]: Local priority. This 3-bit field is used when MCR[LPRI0_EN] is set and makes sense only for TX buffers. If CTRL[LBUF] is set this field is not used. These bits are not transmitted. They are appended to the regular ID to define the transmission priority. Note: Implementation specific Parameter. Not AutoSar Required.	
Screenshot	Can ID Bits Local Priority (0 -> 7) 	
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	Flagchip

3.2.2.2.6 Can Object ID (MB Handle)

Variable	CanObjectId	
Description	Holds the handle ID of HRH or HTH. The value of this parameter is unique in a given CAN Driver, and it should start with 0 and continue without any gaps. The HRH and HTH Ids are defined under two different name-spaces. Example: HRH0-0, HRH1-1, HTH0-2, HTH1-3	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	true
	Origin	AUTOSAR_ECUC
	Range	0-65535

3.2.2.2.7 Can MB Type

Variable	CanObjectType	
Description	Specifies if the HardwareObject is used as Transmit or as Receive object. Note: MBs configured as MIXED standard and RECEIVE type will be treated as EXTENDED.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	RECEIVE
	Origin	AUTOSAR_ECUC
	Range	RECEIVE, TRANSMIT

3.2.2.2.8 CanTriggerTransmitEnable

Variable	CanTriggerTransmitEnable	
Description	This parameter defines if or if not Can supports the trigger-transmit API for this handle.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

3.2.2.2.9 Can Controller Reference

Variable	CanControllerRef	
Description	Reference to CAN Controller to which the HOH is associated to.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

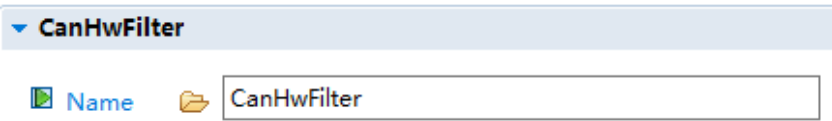
3.2.2.2.10 Can MainFunction RW Period Reference

Variable	CanMainFunctionRWPeriodRef	
Description	Reference to CAN Controller to which the HOH is associated to.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.2.2.11 Can RAM block Reference

Variable	CanRAMBlockRef	
Description	Select RAM block.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	Flagchip

3.2.2.2.12 CanHwFilter

Container	CanHwFilter	
Description	This container is only valid for HRHs and contains the configuration (parameters) of one hardware filter.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.2.12.1 Can Hw Filter Code (STD:0x00-7FF, EXT:0x00-1FFFFFFF)

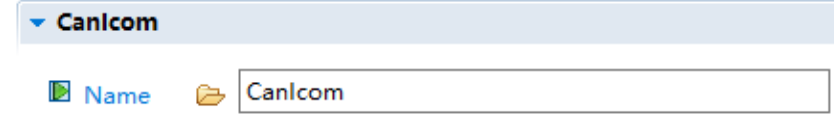
Variable	CanHwFilterCode	
Description	Specifies (together with the filter mask)- the identifiers range that passes the hardware filter for of RX objects. Parameter ranges from 0 to 0x7FF (11 bits) for Standard IDs and 0 to 0x1FFFFFFF (29 bits) for Extended IDs. User can assign any code to this parameter, but must to respect the above rule related to Standard/Extended IDs.	
Screenshot		
Properties	Property	Value
	Type	STRING
	SymbolicName	false
	Default	00
	Origin	AUTOSAR_ECUC

3.2.2.2.12.2 Can Hw Filter Mask (STD:00-7FF, EXT:00-1FFFFFFF)

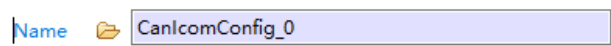
Variable	CanHwFilterMask
-----------------	-----------------

Description	Specifies (together with the filter mask) the identifiers range that passes the hardware filter. EN: Different CanHardwareObjects with different CanIdTypes (STANDARD, MIXED, EXTENDED) can share the same CanFilterMask (i.e., the CanFilterMaskRef parameters of these CanHardwareObjects reference the very same CanFilterMask container). This shall be allowed and must be supported by the configuration generators. The CanFilterMaskRef is omitted for 1) CanHardwareObjects with CanObjectType set to TRANSMIT 2) CanHardwareObjects with CanObjectType set to RECEIVE if only a single Can ID shall be received via this CanHardwareObjects(i.e. exact match with CanIdValue requested when CanHandleType is configured as FULL) ECUC_Can_00321: Reference to the filter mask that is used for hardware filtering together with the CAN_ID_VALUE. This value is used as acceptance masks for ID filtering in RX MBs and the FIFO.	
Screenshot		
Properties	Property	Value
	Type	STRING
	SymbolicName	false
	Default	00
	Origin	AUTOSAR_ECUC

3.2.2.3 CanIcom

Container	CanIcom	
Description	This container contains the parameters for configuring pretended networking	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.3.1 CanIcomConfig

Container	CanIcomConfig	
Description	This container contains the general configuration parameters of the ICOM Configuration	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

3.2.2.3.1.1 Can Icom ID of the configuration

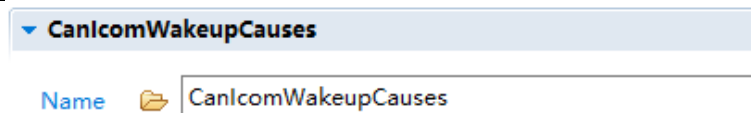
Variable	CanIcomConfigId	
Description	This parameter identifies the ID of the ICOM configuration. In order prevent the issue when have multiple configuration for ICom, please configure the ConfigID follow the order.	
Screenshot		
Properties	Property	Value
	Type	INTEGER

	SymbolicName	false
	Default	1
	Origin	AUTOSAR_ECUC

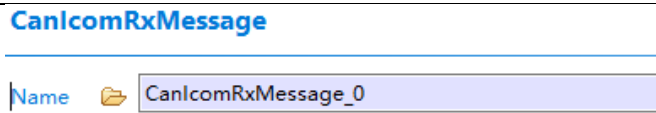
3.2.2.3.1.2 Can Icom wake up busoff detected

Variable	CanIcomWakeOnBusOff	
Description	This parameter defines that the MCU shall wake if the bus off is detected or not.	
Screenshot		
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

3.2.2.3.1.3 CanIcomWakeupCauses

Container	CanIcomWakeupCauses	
Description	This container contains the configuration parameters of the wakeup causes to leave the power saving mode.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

CanIcomRxMessage

Container	CanIcomRxMessage	
Description	This container contains the configuration parameters for the wakeup causes for matching received messages. It has to be configured as often as received messages are defined as wakeup cause. constraint: For all CanIcomRxMessage instances the Message IDs which are defined in CanIcomMessageId and in CanIcomRxMessageIdMask shall not overlap.	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

Can Icom Counter Value

Variable	CanIcomCounterValue	
Description	This parameter defines that the MCU shall wake if the message with the ID is received n times on the communication channel. NOTE: The ASR421 requires 16 bit for this field, but hardware only support 8 bit for this field. So the limitation value of this field is 256.	
Screenshot		

Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	1
	Origin	AUTOSAR_ECUC

Can Icom Message ID

Variable	CanIcomMessageld	
Description	This parameter defines the message ID the wakeup causes of this CanIcomRxMessage are configured for. In addition, a mask (CanIcomMessageldMask) can be defined, in that case it is possible to define a range of RX messages, which can create a wakeup condition.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

Can Icom Message Id Mask

Variable	CanIcomMessageldMask	
Description	Describes a mask for filtering of CAN identifiers. The CAN identifiers of incoming messages are masked with this CanIcomMessageldMask. If the masked identifier matches the masked value of CanIcomMessageld, it can create a wakeup condition for this CanIcomRxMessage. Bits holding a 0 mean don't care, i.e. do not compare the message's identifier in the respective bit position. The mask shall be built by filling with leading 0.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

Can Icom Missing Message Timer

Variable	CanIcomMissingMessageTimerValue	
Description	This parameter defines that the MCU shall wake if the message with the ID is not received for a specific time in s on the communication channel. NOTE: The '0' value have the meaning that the wake-up by timer disable. When you want to disable the wake-up by timer, you should disable this object. The ASR421 don't require about the limitation value for this field, but the hardware only support 16 bit for this field, so this field has the 0-65535 range as the limitation range. The internal timer is incremented based on periodic time ticks, which period is 64 times the CAN Bit Time unit.	
Screenshot		
Properties	Property	Value

	Type	FLOAT
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

Can Icom Payload Length Error

Variable	CanIcomPayloadLengthError	
Description	This parameter defines that the MCU shall wake if a payload error occurs.	
Screenshot	Can Icom Payload Length Error   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	AUTOSAR_ECUC

Can Icom ID Operation

Variable	CanIcomIdOperation	
Description	This is a non-autosar parameter. It is generated in order to support for selection the ID filter type. The Platform supports 4 options in order ID filter wake-up message: EXACTLY SMALLER GREATER INSIDE_RANGE	
Screenshot	Can Icom ID Operation  EXACTLY 	
Properties	Property	Value
	Type	ENUMERATION
	Default	EXACTLY
	Origin	Flagchip
	Range	EXACTLY, GREATER_MINNUM, SMALLER_MAXNUM, INSIDE_RANGE

Can Icom Filter Payload

Variable	CanPayloadFilter	
Description	This parameter defines enable filter payload of messages in Pretended Networking or not	
Screenshot	Can Icom Filter Payload   	
Properties	Property	Value
	Type	BOOLEAN
	SymbolicName	false
	Default	false
	Origin	Flagchip

CanIcomRxMessageSignalConfig

Container	CanIcomRxMessageSignalConfig	
Description	This container contains the configuration parameters for the wakeup causes for matching signals. It has to be configured as often as a signal is defined as wakeup cause. If at least one Signal conditions defined in a	

	CanIcomRxMessageSignalConfig evaluates to true or if no CanIcomRxMessageSignalConfig are defined, the whole wakeup condition is considered to be true. All instances of this container refer to the same frame/pdu (see CanIcomMessageId).	
Screenshot		
Properties	Property	Value
	Type	IDENTIFIABLE

Can Icom Signal Mask

Variable	CanIcomSignalMask	
Description	This parameter shall be used to mask a signal in the payload of a CAN message. The mask is binary AND with the signal payload. The result will be used in combination of the operations defined in CanIcomSignalOperation with the CanIcomSignalValue. the ASR request for full 64 bit with Integers type. but in the Tresos tool, the Integers only has 63 bit, so in the fact, the greatest value is 0x7fffffffffffff.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

Can Icom Signal Operation

Variable	CanIcomSignalOperation	
Description	ECUC_Can_00462: This parameter defines the operation, which shall be used to verify the signal value creates a wakeup condition. The Calypso platform don't support the "AND" filter type.	
Screenshot		
Properties	Property	Value
	Type	ENUMERATION
	SymbolicName	false
	Default	EQUAL
	Origin	AUTOSAR_ECUC
	Range	AND, EQUAL, GREATER, SMALLER, XOR

Can Icom Signal Value

Variable	CanIcomSignalValue	
Description	This parameter shall be used to define a signal value which shall be compared (CanIcomSignalOperation) with the masked CanIcomSignalMask value of the received signal (CanIcomSignalRef).	
Screenshot		
Properties	Property	Value
	Type	INTEGER

	SymbolicName	false
	Default	0
	Origin	AUTOSAR_ECUC

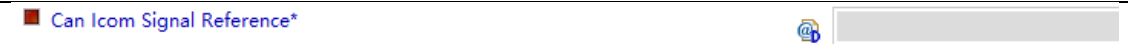
DLC Low Value

Variable	DLCLowValue	
Description	This is a non-autosar object. It is used to configure the lowest value for the "CAN_FLT_DLC" register. That value is number data byte lowest of messages wake-up.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Default	0
	Origin	Flagchip

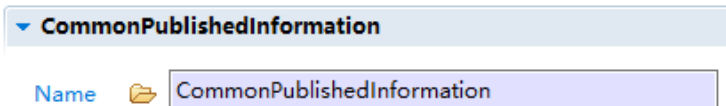
DLC High Value

Variable	DLCHighValue	
Description	This is a non-autosar object. It is used to configure the highest value for the "CAN_FLT_DLC" register. That value is number data byte highest of messages wake-up.	
Screenshot		
Properties	Property	Value
	Type	INTEGER
	Default	0
	Origin	Flagchip

Can Icom Signal Reference

Variable	CanIcomSignalRef	
Description	This parameter defines a reference to the signal which shall be checked additional to the message id (CanIcomMessageId). This reference is used for documentation to define which ComSignal originates this filter setting. All signals being referred by this reference shall point to the same PDU.	
Screenshot		
Properties	Property	Value
	Type	REFERENCE
	Origin	AUTOSAR_ECUC

3.2.3 CommonPublishedInformation

Container	CommonPublishedInformation	
Description	Common container, aggregated by all modules. It contains published information about vendor and versions.	
Screenshot		

Properties	Property	Value
	Type	IDENTIFIABLE

3.2.3.1 Ar Release Major Version

Variable	ArReleaseMajorVersion	
Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	4
	Origin	Flagchip

3.2.3.2 Ar Release Minor Version

Variable	ArReleaseMinorVersion	
Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	6
	Origin	Flagchip

3.2.3.3 Ar Release Revision Version

Variable	ArReleaseRevisionVersion	
Description	Revision version number of AUTOSAR specification on which the appropriate implementation is based on.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	0
	Origin	Flagchip

3.2.3.4 ModuleId

Variable	ModuleId	
Description	Module ID of this module from Module List. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	80

	Origin	Flagchip
--	--------	----------

3.2.3.5 SwMajorVersion

Variable	SwMajorVersion	
Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	0
	Origin	Flagchip

3.2.3.6 SwMinorVersion

Variable	SwMinorVersion	
Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	3
	Origin	Flagchip

3.2.3.7 SwPatchVersion

Variable	SwPatchVersion	
Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	0
	Origin	Flagchip

3.2.3.8 VendorApiInfix

Variable	VendorApiInfix	
Description	In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name. This parameter is used to specify the vendor specific name. In total, the Implementation specific name is generated as follows: <ModuleName>_<VendorId>_<VendorApiInfix><Api name from SWS>. E.g. assuming that the VendorId of	

	the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write. This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1. Note: Implementation Specific Parameter.	
Screenshot		
Properties	Property	Value
	Type	STRING_LABEL
	SymbolicName	false
	Origin	Flagchip

3.2.3.9 VendorId

Variable	VendorId	
Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list. Note: Implementation Specific Parameter	
Screenshot		
Properties	Property	Value
	Type	INTEGER_LABEL
	SymbolicName	false
	Default	174
	Origin	Flagchip

Chapter 4 Configuration Guides

4.1 CAN Usage Common Steps

Basically, the CAN module can be configured by following the below 4 steps:

- 1) Configure CAN general configurations and enable FlexCan Port configuration.
- 2) Configure CAN controller configuration.
- 3) Configure Hardware Object configuration.
- 4) Generate configuration files.

4.2 CAN Usage Demo

- 1) Set the FLEXCAN pin mode in PortContainer firstly.

PortContainer

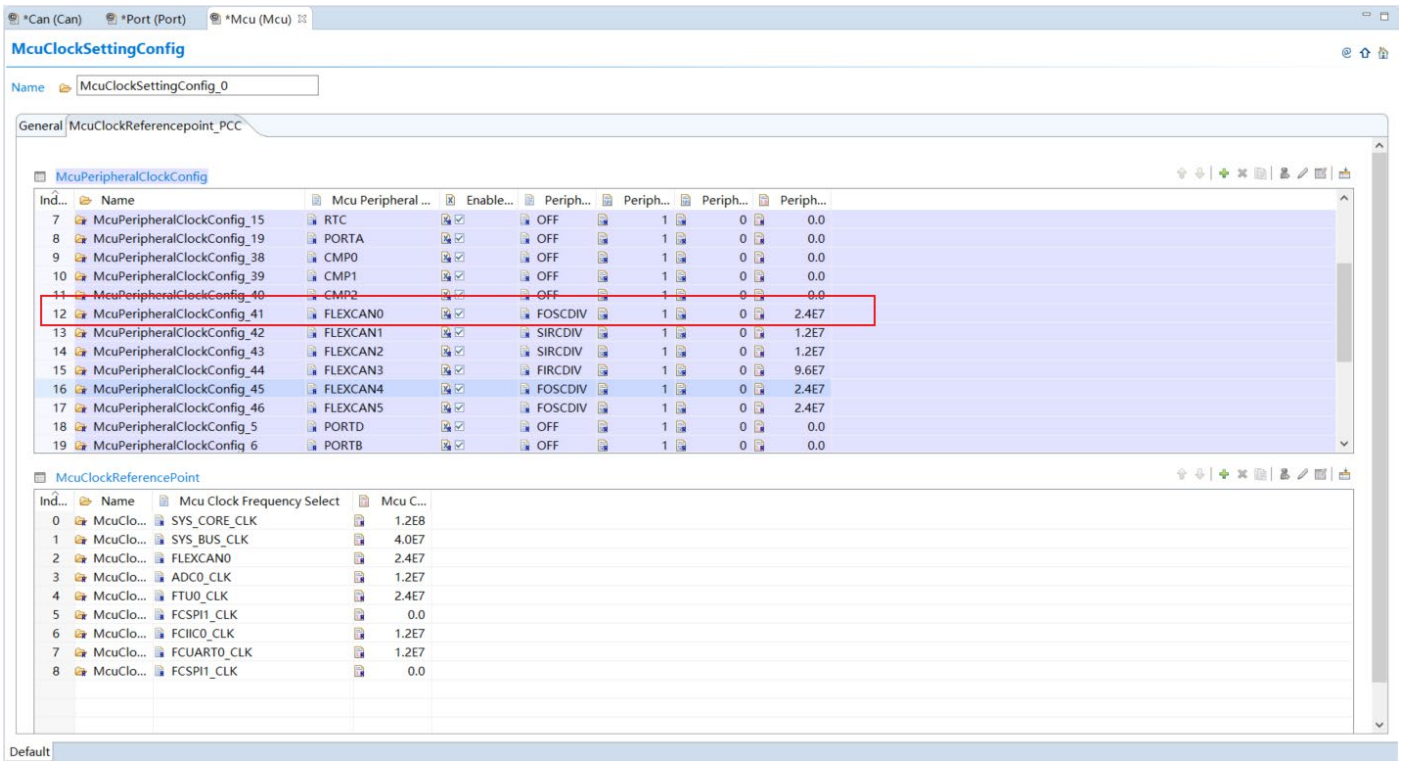
Name: PortContainer_0

General PortPin DigitalFilterChannel

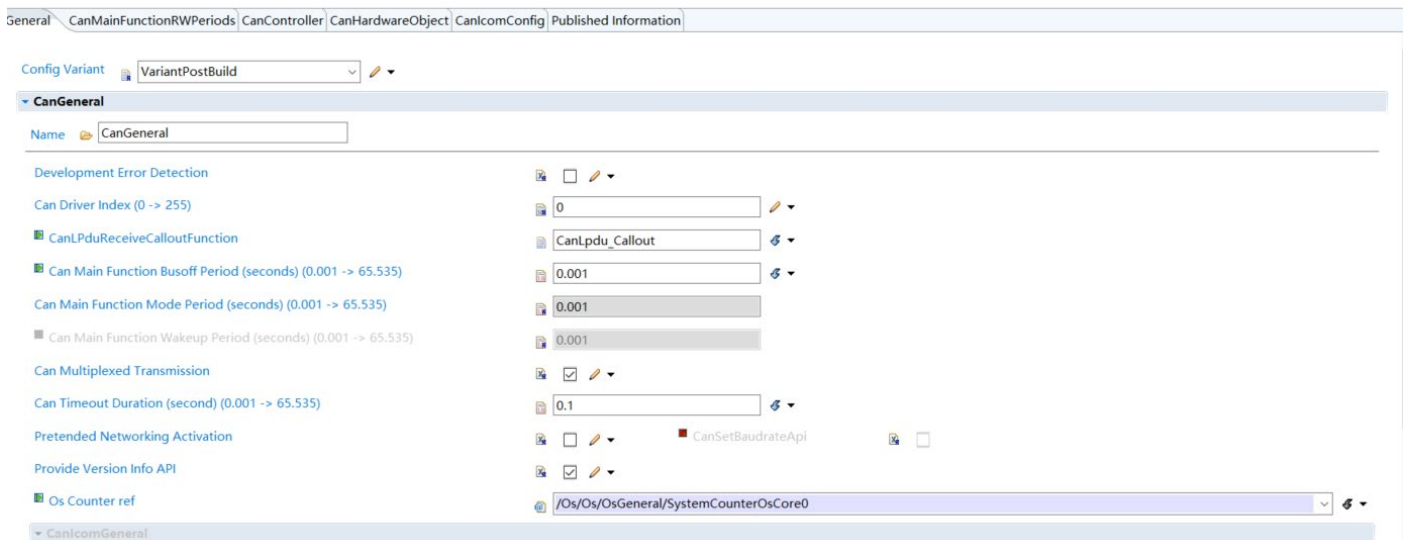
PortPin

Ind...	Name	PortPL...	PortPL...	PortPL...	PortPL...	PortPin Mode	PortPL...	PortPL...	PortPL...	PortPL...	PortPL...	PortPL...	PortPL...	PortPL...	PortPL...	PortPL...	PortPL...
0	PortPin...	1	31	PTA31	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
1	PortPin...	2	30	PTA30	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
2	PortPin...	3	29	PTA29	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
3	PortPin...	4	28	PTA28	☒	FLEXCAN0_RX	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
4	PortPin...	5	27	PTA27	☒	FLEXCAN0_TX	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
5	PortPin...	6	26	PTA26	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
6	PortPin...	7	25	PTA25	☒	ADC0_SE0	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
7	PortPin...	8	17	PTA17	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
8	PortPin...	9	16	PTA16	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
9	PortPin...	10	15	PTA15	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
10	PortPin...	11	14	PTA14	☒	GPIO_TRGSEL_IN13	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
11	PortPin...	12	13	PTA13	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
12	PortPin...	13	12	PTA12	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
13	PortPin...	14	11	PTA11	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
14	PortPin...	15	10	PTA10	☒	JTAG_TDO_TRACE_SWO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
15	PortPin...	16	9	PTA9	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
16	PortPin...	17	8	PTA8	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
17	PortPin...	18	7	PTA7	☒	FCSP11_SCK	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
18	PortPin...	19	6	PTA6	☒	GPIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
19	PortPin...	20	5	PTA5	☒	RESET_b	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			
20	PortPin...	21	4	PTA4	☒	JTAG TMS SWD DIO	☐	Low_dr...	PullDisa...	PullDown	☒	PORT_P...	PORT_...	PORT_P...			

2) Set the FLEXCAN clock in McuClockReferencepoint_PCC container.



3) In CAN General configuration tag, set the busoff and mode main function periods and other configurations. If Det isn't needed, the Development Error Detection checkbox has no need to be checked.



General CanMainFunctionRWPeriods CanController CanHardwareObject CanIcomConfig Published Information

CanIcomGeneral

Name CanIcomGeneral

Level of Pretended Networking CAN_ICOM_LEVEL_ONE

Variant of Pretended Networking CAN_ICOM_VARIANT_NONE

Can define loop as cycle ☐

Can Timeout Duration (in Number of loops) (1 -> 65535) 1

Can Extended ID Support ☒ Can MB Count Extension Support ☒

CanApiEnableMbAbort ☐ CanAbortOnlyOneMB ☒

CanConfigSet

Name CanConfigSet

CanIcom

Name CanIcom

- 4) CanMainFunctionRWPeriods contains the parameter for configuring the period for cyclic call to Can_MainFunction_Read or Can_MainFunction_Write depending on the referring item.

General CanMainFunctionRWPeriods CanController CanHardwareObject CanIcomConfig Published Information

CanMainFunctionRWPeriods

Ind...	Name	CanM...
0	CanMainFunctionRWPeriods_0	0.001

- 5) Create at least one item in CanController. This container contains the configuration parameters of the CAN controller(s).

General CanMainFunctionRWPeriods CanController CanHardwareObject CanIcomConfig Published Information

CanController*

Ind...	Name	Can Hardware ...	Can B...	Can C...	Can C...	Can C...	Can R...	Can Tx...	CanW...	Can W.
0	CanController_0	FlexCAN_0	POLLING	☒	0	0	POLLING	POLLING	☐	POLLING

6) Set Cancontroller general configuration.

General CanControllerBaudrateConfig CanLegacyRxFifoTable

Can Hardware Channel FlexCAN_0

Can BusOff Processing Type POLLING

Can Controller Activation ☒

Can Controller Base Address (0 -> 4294967295) 0

Can Controller ID 0

Can Rx Processing Type POLLING

Can Tx Processing Type POLLING

CanWakeupFunctionalityAPI ☐

Can Wakeup Processing Type POLLING

Can Wakeup Support ☐

Can Controller Default Baudrate /Can/Can/CanConfigSet/CanController_0/CanControllerBaudrateConfig_0

Can CPU Module Reference Clock eConfiguration/McuClockSettingConfig_0/McuClockReferencePoint_Can0

General CanControllerBaudrateConfig CanLegacyRxFifoTable

Can CPU Function Reference Clock eConfiguration/McuClockSettingConfig_0/McuClockReferencePoint_Can0

CanWakeupSourceRef

Can Controller Icom Supported ☐

Can Listen Only Mode (CTRL[LOM]) ☐

Can Enable Software BusOff Recovery ☐

Can Triple Sampling Mode Enable (CTRL[SMP]) ☐

Can Local Priority Enable (MCR[LPRIO_EN]) ☐

Can FD Enable ☒

Can FD Mode* ISO_CAN_FD

Can Legacy RxFifo Enable ☐

Can Legacy Message buffer data size(FDCTRL[MBDSR0]) CAN_8_BYTES_PAYLOAD

Can Legacy Maximum Message Buffers 21

CanRxFifoWarningNotification NULL_PTR

CanRxFifoOverflowNotification NULL_PTR

Can BCC Enable ☒

Can Loop Back Mode (CTRL[LPB]) ☐

Can Auto BusOff Recovery (CTRL[BOFF_REC]) ☐

Can Lowest Buffer Transmit First (CTRL[LBUF]) ☒

Can Clock from Bus (CTRL[CLK_SRC]) ☐

General CanControllerBaudrateConfig CanLegacyRxFifoTable

Can Legacy RxFifo Enable ☐

Can Legacy Message buffer data size(FDCTRL[MBDSR0]) CAN_8_BYTES_PAYLOAD

Can Legacy Maximum Message Buffers 21

CanRxFifoWarningNotification NULL_PTR

CanRxFifoOverflowNotification NULL_PTR

Can Error Notification Enable ☒

CanErrorControllerNotification NULL_PTR

CanErrorFDControllerNotification NULL_PTR

Can Legacy RxFifo

Name CanLegacyRxFifo

Can Legacy RxFifo ID Acceptance Mode A

Can Legacy RxFifo Filters Number FILTERS_NUMBER_8

Can Legacy RxFifo Global Filter Mask (0 -> 4294967295) 2047

7) Set baudrate.

General

Can Module Clock (Hz) - Reselect after clock change

24000000

500

0

3

16

1

7

4

4

4

0.7333333333333333

20

CanControllerFdBaudrateConfig

- 8) Create objects in CanHardwareObjects. This container contains the configuration (parameters) of CAN Hardware Objects. This configuration element is used as information for the CAN Interface only.

General

CanMainFunctionRWPeriods

CanController

CanHardwareObject

CanComConfig

Published Information

CanHardwareObject

Ind...	Name	Can I...	Can H...	Can ID...	Can ID...	Can O...	Can M...	Can Controller Reference	Can
0	CanHardwareObject_0	BASIC	1	STAND...	0	0	RECEIVE	/Can/Can/CanConfigSet/Ca...	0x7df
1	CanHardwareObject_1	BASIC	1	STAND...	0	1	RECEIVE	/Can/Can/CanConfigSet/Ca...	0x785
2	CanHardwareObject_2	BASIC	1	STAND...	0	2	RECEIVE	/Can/Can/CanConfigSet/Ca...	0x400
3	CanHardwareObject_3	FULL	1	EXTEN...	0	3	RECEIVE	/Can/Can/CanConfigSet/Ca...	0x22
4	CanHardwareObject_4	BASIC	1	STAND...	0	4	TRANS...	/Can/Can/CanConfigSet/Ca...	00
5	CanHardwareObject_5	BASIC	1	STAND...	0	5	TRANS...	/Can/Can/CanConfigSet/Ca...	00
6	CanHardwareObject_6	BASIC	1	STAND...	0	6	TRANS...	/Can/Can/CanConfigSet/Ca...	00

9) Set the HardwareObject configurations.

- The receive hardware object can be set by following the configuration below.

General

FD padding value (0 -> 255)

Can Implementation Type

Can Hardware Object Count (1 -> 65535)

Can ID Message Type

Can ID Bits Local Priority (0 -> 7)

Can Object ID (MB Handle)

Can MB Type

CanTriggerTransmitEnable ☐

Can Controller Reference

Can MainFunction RW Period Reference

CanHwFilter

Name

Can Hw Filter Code (STD:0x00-7FF, EXT:0x00-1FFFFFFF)

Can Hw Filter Mask (STD:00-7FF, EXT:00-1FFFFFFF)

- The transmit hardware object can be set by following the configuration below.

General

FD padding value (0 -> 255)

Can Implementation Type

Can Hardware Object Count (1 -> 65535)

Can ID Message Type

Can ID Bits Local Priority (0 -> 7)

Can Object ID (MB Handle)

Can MB Type

CanTriggerTransmitEnable ☒

Can Controller Reference

Can MainFunction RW Period Reference

CanHwFilter

Name

Can Hw Filter Code (STD:0x00-7FF, EXT:0x00-1FFFFFFF)

Can Hw Filter Mask (STD:00-7FF, EXT:00-1FFFFFFF)

10) Click the **Generate Project** to generate relevant code.

